Efficient (Functional) Bootstrapping for CKKS over the Conjugate-Invariant Ring

Jiaming Liu, Shihe Ma, Anyu Wang and Xiaoyun Wang

Tsinghua University, Beijing, China

liujm25@mails.tsinghua.edu.cn msh24@mails.tsinghua.edu.cn
anyuwang@tsinghua.edu.cn xiaoyunwang@tsinghua.edu.cn

Abstract

The CKKS fully homomorphic encryption scheme enables efficient encrypted computation over complex-valued data. In many real-world cases, however, the computation only involves real data, and storing them inside complex slots wastes the encoding ability. This motivated the design of conjugate-invariant CKKS (CI-CKKS) instantiated over the conjugate-invariant ring (CIR), which provides the same number of real slots while halving the RLWE dimension. Despite its better efficiency in handling real data, CI-CKKS cannot be bootstrapped directly in CIR, since the CoeffToSlot and SlotToCoeff linear transformations used in bootstrapping require storing complex values in both plaintexts and ciphertexts. Existing methods bootstrap CI-CKKS ciphertexts via conversion to standard CKKS, perform bootstrapping there, and convert back. For example, Lattigo bootstraps two CI-CKKS ciphertexts by combining them into a CKKS ciphertext with doubled ring dimension, while Bae et al. convert a single CI-CKKS ciphertext to a CKKS ciphertext in the same dimension. The former is efficient only for batched inputs and suffers from limited flexibility, while the latter consumes additional layers in packing. Another issue with CI-CKKS is that many existing functional bootstrapping methods encode intermediate values using complex roots of unity, whereas CI-CKKS provides only real-valued slots. Our first contribution is a real FFT algorithm over the conjugate-invariant ring, thereby enabling native bootstrapping in CI-CKKS. Our second contribution investigates functional bootstrapping over CI-CKKS. We adapt the real-valued AKP interpolant with radial correction and develop a single-cosine Hermite interpolation method using a single real ciphertext. We show existing complex-valued interpolation methods can be implemented using two real ciphertexts to simulate complex arithmetic. We also study the numerical behavior of real-valued interpolation and evaluate the tradeoffs between computational efficiency and precision. We implement our methods in Lattigo. Our native CI-CKKS bootstrapping achieves a 1.14–12.86× speedup in single-ciphertext bootstrapping across different baseline methods and total linear transformation depths. Compared with state-of-the-art functional bootstrapping methods over standard CKKS, our methods achieve 1.16–1.68× higher throughput.

Keywords: Fully homomorphic encryption, bootstrapping, conjugate-invariant ring, CKKS, functional bootstrapping

1 Introduction

Fully homomorphic encryption (FHE) enables computation on encrypted data without revealing the underlying plaintexts [Gen09]. The CKKS scheme [CKKS17] supports approximate single-instruction-multiple-data (SIMD) arithmetic over encrypted vectors of

complex numbers. However, many FHE applications, including privacy-preserving machine learning [BCCW19, AAB+23, EGR25, CWCW25, Rah22, PPP+26] and CKKS-based AES transciphering [NHY+25, BCKK25, KLN+26], involve computations on real-valued or discrete data. Under conventional real-valued encoding, the imaginary parts of CKKS slots remain unused. To obtain real-valued slots, Kim and Song [KS19] proposed a CKKS variant over the conjugate-invariant ring (CIR). For the same ring rank, their scheme supports twice as many real slots as standard CKKS with comparable computational costs, leading to almost doubled throughput in computation.

Bootstrapping refreshes ciphertexts by homomorphically evaluating the decryption function, and is the key to evaluating circuits of arbitrary depth [Gen09]. For CKKS, bootstrapping increases the ciphertext modulus while approximately preserving the encrypted message [CHK+18]. It involves modulus raising, CoeffToSlot linear transformation, approximate modular reduction, and SlotToCoeff. CKKS bootstrapping has undergone extensive research in the past years, including advancements in more efficient and accurate modular reduction [CCS19, LLK+22, JM22, KPK+22, KN25], faster linear transformations [CHH18, CCS19, BMTH21, BCH+24], higher bootstrapping precision [BCC+22, CKSS25, BFS+26], and new bootstrapping circuits [CHKS25, CK25, CS25]. However, there are few studies for bootstrapping CKKS in CIR, and many existing CKKS bootstrapping methods are not designed for direct evaluation over the conjugate-invariant ring.

The first problem in CI-CKKS bootstrapping is the lack of a CIR-native CoeffToSlot and SlotToCoeff. In standard CKKS, the sparse FFT factorization of CoeffToSlot and SlotToCoeff is essential for their efficiency [CHH18, CCS19]. However, the twiddle factors and intermediate FFT results are complex, so they cannot be represented by CI-CKKS ciphertexts. Existing methods bootstrap conjugate-invariant ciphertexts by converting them to standard CKKS and converting the results back. For example, the Lattigo library packs two CI-CKKS ciphertexts into a CKKS ciphertext with doubled ring dimension, where the two real-valued CI-CKKS slots are stored into the real and imaginary parts of the complex CKKS slots [Tun26]. The efficiency of this bootstrapping relies on amortizing its cost over the two input ciphertexts, which limits its flexibility in practice. Alternatively, Bae et al. [BCKK25] proposed packing the N real CI-CKKS slots into the $N/2$ CKKS slots via tailored conversion algorithms, ensuring that the CKKS ciphertext has the same ring dimension as the CI-CKKS ciphertext. This method provides better flexibility than the Lattigo method, but consumes two additional levels during the conversions between CI-CKKS and CKKS. Kim et al. [KLN+26] further discussed how to avoid this additional layer consumption by performing the linear transformations in a ring with dimension $2N$, at the cost of additional computational overhead.

The second problem is that existing functional bootstrapping methods for discrete CKKS [DMPS24] use complex-valued data during their computation, which cannot be natively supported by CI-CKKS. Previous works have developed bootstrapping for bits and small integers [BCKS24, BKSS24], lookup-table evaluation [CKKL24], and general functional bootstrapping that evaluates lookup functions while reducing the input noise [DAP+26, AKP25, AKPZ26]. However, the state-of-the-art methods map discrete values into complex roots of unity to enable efficient homomorphic evaluation [AKP25, BKSS24, AKPZ26], which does not naturally extend to the real-valued computation in CI-CKKS. Alternatively, Alexandru et al. studied the direct evaluation of real-valued Hermite interpolation using sine and cosine and showed that approximation errors can cause numerical instability [AKPZ26]. In particular, they identified native bootstrapping over the conjugate-invariant ring and efficient real-valued Hermite interpolation as open problems.

1.1 Our Contributions

We resolve the former problem and explore several approaches to the latter, thereby enhancing the efficiency and usability of CI-CKKS.

Our first contribution is a sparse real matrix factorization for CoeffToSlot and SlotToCoeff over the conjugate-invariant ring. Starting from the standard FFT decomposition, we exploit conjugate symmetry to replace the complex factor matrices with real matrices. The resulting decomposition consists of sparse orthogonal matrices, where each matrix has at most seven diagonals. The same level-collapsing and BSGS techniques in existing literature [CHH18, CCS19, HS18] also apply to our method.

Our second contribution is functional bootstrapping over the conjugate-invariant ring. We adapt the real-valued Fourier–Hermite interpolation of AKP [AKP25] to CI-CKKS with an efficient radial correction. We also construct a Hermite interpolant as a real polynomial in a single cosine variable, enabling evaluation from one real-valued ciphertext. Furthermore, we show that existing complex-valued interpolation methods can be implemented over CI-CKKS by simulating complex arithmetic, using Sparse-THI [AKPZ26] as an example. We analyze the computational complexity and compare the latency, precision, and noise capacity of these methods.

We implement both constructions in Lattigo [Tun26]. For a single CI ciphertext, our native CI-CKKS bootstrapping achieves speedups of 1.50–1.86×, 1.14–12.86×, and 1.17–9.64× over Lattigo, Bae et al., and Kim et al., respectively, across different total linear transformation depths. Even compared with the standard CKKS configuration that bootstraps two CI ciphertexts together, our method achieves a lower amortized latency at $L = 7$ (12.09 versus 13.25 seconds per CI ciphertext), without requiring two input ciphertexts to be batched. For functional bootstrapping at the same ring dimension, CI-AKP and CI-Hermite achieve 1.22–1.40× and 1.48–1.68× the throughput of AKP over standard CKKS, respectively. CI-Sparse-THI achieves 1.16–1.43× the throughput of Sparse-THI over standard CKKS.

1.2 Technical Overview

We now describe the main ideas underlying our constructions.

Real DFT Matrix Factorization. We view the iDFT in the rank-N conjugate invariant ring as a real-input iDFT over the N complex slots in the standard cyclotomic ring. Following the standard complex DFT, we observe that the intermediate values during a real-input iDFT has a *skewed conjugate-symmetry*. Specifically, after applying i iDFT layers, we partition the resulting vector into $m = 2^i$ sub-vector with equal length

$$\widetilde{\mathbf{z}}_{\mathbf{m}} = (\widetilde{\mathbf{z}}_{m,0} | \widetilde{\mathbf{z}}_{m,1} | \dots | \widetilde{\mathbf{z}}_{m,m-1}).$$

These sub-vectors satisfy the skewed conjugate-symmetry

$$(\widetilde{\mathbf{z}}_{m,m-r})_j = (\overline{\widetilde{\mathbf{z}}_{m,r}})_j \cdot \exp(-m5^j \pi i/(2N)).$$

By tweaking $\widetilde{\mathbf{z}}_m$ with proper factors, we obtain sub-vectors $\mathbf{z}_{m,r}$ with perfect conjugate symmetry

$$\mathbf{z}_{m,m-r} = \overline{\mathbf{z}_{m,r}}, \qquad 0 \le r < m.$$

This immediately implies that $\mathbf{z}_{m,0}$ and $\mathbf{z}_{m,m/2}$ are real, while other sub-vectors appear in conjugate pairs. Therefore, each conjugate pair can be replaced by two real vectors storing their real and imaginary parts. This makes all intermediate iFFT values real. The iFFT matrices are also real because their outputs remain real-valued for arbitrary real inputs. We therefore successfully eliminate the usage of complex values throughout the

real-input iDFT, making it possible to be evaluated directly in the conjugate-invariant ring.

It only remains to make these iFFT matrices diagonally sparse, so that they can be efficiently evaluated in homomorphic computation. The standard complex iFFT introduces recurrence relationships

$$\begin{aligned}(\mathbf{z}_{2m,r})_j &= (\mathbf{z}_{m,r})_j + \omega_{m,j,r}(\mathbf{z}_{m,r})_{j+N/(2m)},\\ (\mathbf{z}_{2m,r+m})_j &= (\mathbf{z}_{m,r})_j - \omega_{m,j,r}(\mathbf{z}_{m,r})_{j+N/(2m)},\end{aligned} \qquad 0 \le r < m,\ 0 \le j < N/(2m)$$

where $\omega_{m,j,r}$ is a complex root of unity. Let $L = N/(2m)$, these relationships can be converted into real ones as follow.

- **Real to real**. Setting $r = 0$ gives the relationship between the real vectors $(\mathbf{z}_{2m,0}, \mathbf{z}_{2m,m})$ and $\mathbf{z}_{m,0}$.
- **Real to complex**. The other real input vector $\mathbf{z}_{m,m/2}$ produces $\mathrm{Re}(\mathbf{z}_{2m,m/2})$ and $\mathrm{Im}(\mathbf{z}_{2m,m/2})$ via

$$\begin{aligned}\mathrm{Re}(\mathbf{z}_{2m,m/2})_j &= (\mathbf{z}_{m,m/2})_j\\ \mathrm{Im}(\mathbf{z}_{2m,m/2})_j &= \mathrm{Im}(\omega_{m,j,r})(\mathbf{z}_{m,m/2})_{j+L}.\end{aligned}$$

- **Complex to complex**. For all $0 < r < m/2$, each input pair $(\mathrm{Re}(\mathbf{z}_{m,r}), \mathrm{Im}(\mathbf{z}_{m,r}))$ produces two output pairs $(\mathrm{Re}(\mathbf{z}_{2m,r}), \mathrm{Im}(\mathbf{z}_{2m,r}), \mathrm{Re}(\mathbf{z}_{2m,m-r}), \mathrm{Im}(\mathbf{z}_{2m,m-r}))$.

$$\begin{aligned}\mathrm{Re}(\mathbf{z}_{2m,r})_j &= \ \ \mathrm{Re}(\mathbf{z}_{m,r})_j + \alpha\mathrm{Re}(\mathbf{z}_{m,r})_{j+L} - \beta\mathrm{Im}(\mathbf{z}_{m,r})_{j+L}\\ \mathrm{Im}(\mathbf{z}_{2m,r})_j &= \ \ \mathrm{Im}(\mathbf{z}_{m,r})_j + \beta\mathrm{Re}(\mathbf{z}_{m,r})_{j+L} + \alpha\mathrm{Im}(\mathbf{z}_{m,r})_{j+L}\\ \mathrm{Re}(\mathbf{z}_{2m,m-r})_j &= \ \ \mathrm{Re}(\mathbf{z}_{m,r})_j - \alpha\mathrm{Re}(\mathbf{z}_{m,r})_{j+L} + \beta\mathrm{Im}(\mathbf{z}_{m,r})_{j+L}\\ \mathrm{Im}(\mathbf{z}_{2m,m-r})_j &= -\mathrm{Im}(\mathbf{z}_{m,r})_j + \beta\mathrm{Re}(\mathbf{z}_{m,r})_{j+L} + \alpha\mathrm{Im}(\mathbf{z}_{m,r})_{j+L}.\end{aligned}$$

  The last two equalities use $\mathbf{z}_{2m,r+m} = \overline{\mathbf{z}_{2m,m-r}}$ and $\omega_{m,j,r} = \alpha + i\beta$.

As an example, Figure 1 displays the third iFFT layer, which is the first layer that includes all three types of recurrence relationships.

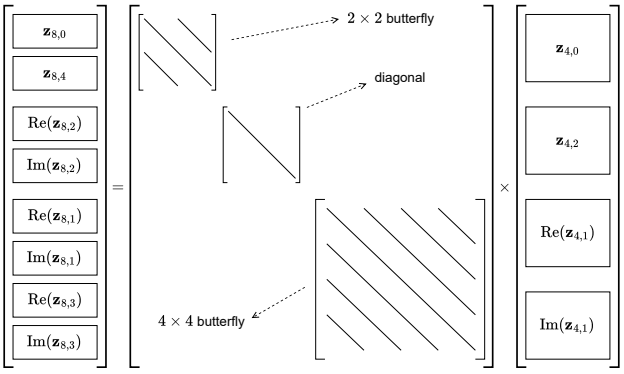


Figure 1: Data arrangement and the sparsity pattern in the third real iFFT layer.

Applying this construction recursively, we obtain

$$\mathbf{P}\mathbf{V}^{-1} = \mathbf{S}\mathbf{F}_\ell \cdots \mathbf{F}_1, \qquad \ell = \log_2 N,$$

where each $\mathbf{F}_i$ is a sparse real orthogonal matrix with at most seven diagonals, $\mathbf{S}$ is a diagonal scaling matrix, and $\mathbf{P}$ is a permutation matrix determined by the coefficient order. Since the matrices $\mathbf{F}_i$ are orthogonal, the reverse transformation is given by

$$\mathbf{V}\mathbf{P}^{\mathsf{T}} = \mathbf{F}_1^{\mathsf{T}} \cdots \mathbf{F}_\ell^{\mathsf{T}}\mathbf{S}^{-1}.$$

Likewise to standard CKKS bootstrapping, $\mathbf{P}$ and its inverse need not be homomorphically evaluated. The same level collapsing technique [CHH18, CCS19] can still be adopted to provide a tradeoff between level consumption and computation cost.

Native bootstrapping for conjugate-invariant CKKS. Our real DFT matrix factorization enables native CKKS bootstrapping over the conjugate-invariant ring. We replace the StC and CtS transformations with our real linear transformations, while retaining ModRaise and EvalMod. Unlike previous methods, which convert conjugate-invariant ciphertexts to a standard CKKS ring, our method performs bootstrapping directly over the conjugate-invariant ring. This removes the need for input batching or additional multiplication levels in existing methods, at the cost of an increased number of nonzero diagonals in the FFT matrices.

Functional bootstrapping in CI-CKKS. We further develop functional bootstrapping for discrete messages in CI-CKKS. For a public lookup function $f : \mathbb{Z}_p \to \mathbb{Z}_p$, we first apply StC, ModRaise, and CtS using our real linear transformations. The resulting slots have the form $x = I + m/p + v$, where $m \in \mathbb{Z}_p$, $I \in \mathbb{Z}$ is introduced by modulus raising, and $v \in \mathbb{R}$ is the input error. Our goal is to remove I and evaluate $f(m)/p$ while reducing the input error.

A straightforward approach is to emulate complex arithmetic using two CI-CKKS ciphertexts, which separately encrypt the real and imaginary components of a complex value. For example, multiplying two complex values requires only three real ciphertext multiplications:

$$(a + ib)(d + ie) = (ad - be) + i((a + b)(d + e) - ad - be).$$

This allows existing complex-valued interpolation methods, including AKPZ [AKPZ26], to be applied directly over CI-CKKS.

We further investigate interpolation methods that operate directly on real values. A natural choice is the real-valued Fourier–Hermite interpolant of AKP [AKP25], which can be rewritten using Chebyshev identities as

$$R(x) = A(\cos(2\pi x)) + \sin(2\pi x)B(\cos(2\pi x)),$$

where A and B are real polynomials determined by the Fourier coefficients. This representation allows us to evaluate the interpolant using two real-valued ciphertexts that approximate the cosine and sine functions.

However, as observed in Appendix A of AKPZ [AKPZ26], independently approximating cosine and sine may introduce additionsl errors. Let $c \approx \cos(2\pi x)$ and $s \approx \sin(2\pi x)$ denote the computed values, which generally satisfy $c^2 + s^2 = 1 + \varepsilon'$ rather than 1. To see the effect of this deviation, let $c_0 = \cos(2\pi x)$ and $s_0 = \sin(2\pi x)$ denote the exact values and consider a radial perturbation $(c, s) = \sqrt{1 + \varepsilon'}(c_0, s_0)$. A first-order Taylor expansion gives

$$A(c) + sB(c) = R(x) + \frac{\varepsilon'}{2}\big(c_0A'(c_0) + s_0B(c_0) + c_0s_0B'(c_0)\big) + O((\varepsilon')^2).$$

Although the Hermite conditions ensure $R(m/p + v) = f(m)/p + O(|v|^2)$, they do not generally eliminate the first-order radial error. Consequently, the evaluated result may contain an additional $O(|\varepsilon'|)$ error that dominates the quadratic noise reduction of AKP.

To address this issue, we incorporate a correction into the final double-angle step. AKPZ [AKPZ26] also proposed a correction method, called AKPnorm, which normalizes the complex exponential after EvalExp using an inverse-square-root approximation. In contrast, our correction is incorporated into the final double-angle step, allowing us to reuse the ciphertext squares computed in this step.

Let $c' \approx \cos(\pi x)$ and $s' \approx \sin(\pi x)$ denote the approximations before this step, with $c'^2 + s'^2 = 1 + \varepsilon$. The ordinary double-angle formulas give $c = c'^2 - s'^2$ and $s = 2c's'$, so $c^2 + s^2 = (1+\varepsilon)^2$ instead of 1. Since ε is encrypted, division by $1+\varepsilon$ cannot be performed directly in CKKS. We therefore use the first-order Taylor approximation $1/(1+\varepsilon) \approx 1 - \varepsilon$ and compute

$$C = c(1-\varepsilon), \qquad S = s(1-\varepsilon).$$

Since $(1+\varepsilon)(1-\varepsilon) = 1-\varepsilon^2$, the corrected values satisfy $\sqrt{C^2+S^2} = 1-\varepsilon^2$ for sufficiently small ε, reducing the error in the magnitude from first to second order. Higher-order error reduction can be achieved similarly by using higher-degree approximations of $1/(1+\varepsilon)$.

The correction reuses the two ciphertext squares from the final double-angle step to compute $\varepsilon = c'^2 + s'^2 - 1$. For $n = 1$, it requires only two additional ciphertext multiplications and one multiplicative level. For higher orders, the geometric series can be evaluated with $O(\log(n+1))$ additional multiplicative levels.

We also construct a univariate real-valued Hermite interpolant using $u(x) = \cos(2\pi x + \pi/(2p))$. The values $u_m = u(m/p)$ for $m \in \mathbb{Z}_p$ are precisely the p distinct roots of the Chebyshev polynomial T_p. We construct a real polynomial H of degree at most $(n+1)p-1$ satisfying $H(u_m) = f(m)/p$ and $H^{(j)}(u_m) = 0$ for $1 \le j \le n$. By periodicity, $H(u(x))$ removes the integer overflow and reduces the input error to $O(|v|^{n+1})$. The interpolant can be evaluated using a single real-valued CI-CKKS ciphertext.

2 Preliminaries

Notation. Let $N \ge 2$ be a power of two and $Q \ge 2$ an integer. We define $\widetilde{\mathcal{R}}_N = \mathbb{Z}[X]/(X^N+1)$ and its maximal real subring $\mathcal{R}_{N/2} = \mathbb{Z}[X + X^{-1}] \subseteq \widetilde{\mathcal{R}}_N$. We write $\widetilde{\mathcal{R}}_{N,Q} = \widetilde{\mathcal{R}}_N / Q\widetilde{\mathcal{R}}_N$ and $\mathcal{R}_{N,Q} = \mathcal{R}_N / Q\mathcal{R}_N$, and omit subscripts when clear from context. We write $\zeta_N = \exp(2\pi i/(N))$ for a primitive N-th root of unity. Vectors and matrices are denoted by bold lowercase and uppercase letters, respectively, and all vectors are column vectors. For a vector $\mathbf{a}$, we write a_i for its i-th entry, and for a matrix $\mathbf{A}$, we write $A_{i,j}$ for its (i,j)-th entry and $\mathbf{A}^\mathsf{T}$ for its transpose. The symbol $\odot$ denotes componentwise multiplication. For a vector $\mathbf{a}$, we write $\|\mathbf{a}\|_\infty = \max_i |a_i|$. All logarithms are to base two unless otherwise stated. For an integer $q \ge 2$, let $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$.

2.1 The CKKS Scheme

The CKKS scheme [CKKS17] supports approximate homomorphic arithmetic on vectors of complex numbers.

Encoding. CKKS supports coefficient encoding and slot encoding. For $\mathbf{z} \in \mathbb{R}^N$, coefficient encoding stores the scaled and rounded entries of $\mathbf{z}$ as polynomial coefficients:

$$\mathsf{Ecd}_{\mathrm{coeff}}(\mathbf{z}; \Delta) = \sum_{i=0}^{N-1} \lfloor \Delta z_i \rceil X^i \in \widetilde{\mathcal{R}}_N.$$

Slot encoding uses the canonical embedding $\mathsf{DFT}(m) = (m(\zeta_{2N}^{5^i}))_{0 \le i < N/2}$ and its inverse iDFT. For $\mathbf{z} \in \mathbb{C}^{N/2}$, slot encoding computes

$$\mathsf{Ecd}_{\mathrm{slot}}(\mathbf{z}; \Delta) = \lfloor \Delta \cdot \mathsf{iDFT}(\mathbf{z}) \rceil \in \widetilde{\mathcal{R}}_N,$$

where rounding is applied coefficientwise. Decoding applies DFT and divides by Δ. The homomorphic transformations CtS and StC convert between coefficient encoding and slot encoding.

To evaluate these transformations efficiently, Cheon et al. [CHH18] and Chen et al. [CCS19] decomposed the corresponding DFT matrices into sparse factors, each with a constant number of nonzero diagonals. To reduce the multiplicative depth, $\log_2 r$ consecutive factors can be combined into a matrix with $O(r)$ nonzero diagonals, where r is a power of two. Using the diagonal method with BSGS, each combined matrix requires $O(\sqrt{r})$ rotations and $O(r)$ plaintext–ciphertext multiplications. For $N/2$ slots, the total cost is $O(\sqrt{r}\log_r(N/2))$ rotations and $O(r\log_r(N/2))$ plaintext–ciphertext multiplications, with multiplicative depth $O(\log_r(N/2))$.

Encryption and Operations. A CKKS ciphertext $\mathsf{ct} = (a, b) \in \widetilde{\mathcal{R}}^2_{N,Q}$ encrypting a plaintext polynomial $m \in \widetilde{\mathcal{R}}_N$ under a secret key $s \in \widetilde{\mathcal{R}}_N$ satisfies

$$b + as = m + e \pmod{Q},$$

where e is a small error polynomial. For a ternary secret key s, its Hamming weight is $\mathrm{hwt}(s) = \|s\|_1$.

CKKS supports homomorphic addition (Add), subtraction (Sub), plaintext multiplication (PtMult), ciphertext multiplication (Mult), key switching (KeySwitch), rescaling (Rescale), slot rotation (Rot), and conjugation (Conj). Ciphertext multiplication uses relinearization with $\mathsf{KSK}_{s^2\to s}$, while rescaling reduces the ciphertext modulus and scale. Slot rotation by k positions applies the automorphism $X \mapsto X^{5^k}$ followed by key switching, and conjugation applies $X \mapsto X^{-1}$ followed by key switching.

Homomorphic linear transformations. Let $\mathbf{A} \in \mathbb{C}^{(N/2)\times(N/2)}$ and $\mathbf{z} \in \mathbb{C}^{N/2}$. For $0 \le k < N/2$, define the k-th cyclic diagonal $\mathbf{a}_k$ by $(\mathbf{a}_k)_i = A_{i,(i+k) \bmod (N/2)}$. The diagonal method computes

$$\mathbf{Az} = \sum_{k=0}^{N/2-1} \mathbf{a}_k \odot \mathsf{Rot}_k(\mathbf{z}).$$

Each nonzero diagonal requires one plaintext–ciphertext multiplication and, for $k \neq 0$, one rotation. Sparse matrix factorizations and BSGS reduce the cost of the linear transformations used in CKKS bootstrapping [CHH18, CCS19].

2.2 CKKS over the Conjugate-Invariant Ring

The conjugate-invariant variant of CKKS supports real-valued slots, with security based on the hardness of the RLWE problem over the conjugate-invariant ring [KS19].

Element Representation. The conjugate-invariant ring $\mathcal{R}_N \subseteq \widetilde{\mathcal{R}}_{2N}$ has rank N over $\mathbb{Z}$. Every $a \in \mathcal{R}_N$ can be uniquely written as

$$a(X) = a_0 + \sum_{j=1}^{N-1} a_j(X^j + X^{-j}), \qquad a_j \in \mathbb{Z},$$

where $X^{-j} = -X^{2N-j}$ in $\widetilde{\mathcal{R}}_{2N}$. We denote its coefficient vector by $\mathbf{a} = (a_0, \ldots, a_{N-1})^\mathsf{T}$.

Encoding. The canonical embedding of $\mathcal{R}_N$ has N real slots. Since $(\mathbb{Z}/4N\mathbb{Z})^\times/\{\pm 1\}$ is generated by 5, we define $\sigma(a) = (a(\zeta_{4N}^{5^i}))_{0\le i<N} \in \mathbb{R}^N$. With respect to the coefficient basis above, the canonical embedding is represented by $\mathbf{V} \in \mathbb{R}^{N\times N}$, where

$$\sigma(a) = \mathbf{Va}, \qquad V_{i,0} = 1, \qquad V_{i,j} = \zeta_{4N}^{5^i j} + \zeta_{4N}^{-5^i j} = 2\cos\left(\frac{\pi 5^i j}{2N}\right) \quad (1 \le j < N). \tag{1}$$

The columns of $\mathbf{V}$ are orthogonal, with squared norms N for the first column and $2N$ for the remaining columns. Hence,

$$\mathbf{V}^{\mathsf{T}}\mathbf{V} = \mathrm{diag}(N, 2N, \ldots, 2N), \qquad \mathbf{V}^{-1} = \mathrm{diag}\left(\frac{1}{N}, \frac{1}{2N}, \ldots, \frac{1}{2N}\right)\mathbf{V}^{\mathsf{T}}. \tag{2}$$

For $\mathbf{z} \in \mathbb{R}^N$, encoding at scale Δ rounds $\Delta\mathbf{V}^{-1}\mathbf{z}$ coefficientwise to obtain a plaintext in $\mathcal{R}_N$. Decoding applies $\mathbf{V}$ to the plaintext coefficient vector and divides by Δ.

Encryption. A CI-CKKS ciphertext $\mathsf{ct} = (a, b) \in \mathcal{R}_{N,Q}^2$ encrypting a plaintext polynomial $m \in \mathcal{R}_N$ under a secret key $s \in \mathcal{R}_N$ satisfies

$$b + as = m + e \pmod{Q},$$

where $e \in \mathcal{R}_N$ is a small error polynomial. For a ternary secret key s, each coefficient belongs to $\{-1, 0, 1\}$, and its Hamming weight is $\mathrm{hwt}(s) = \|s\|_1$.

Homomorphic operations. CI-CKKS supports the same homomorphic operations as standard CKKS, with all computations performed over $\mathcal{R}_{N,Q}$. Let $\mathsf{ct}_i = (a_i, b_i)$ denote a ciphertext under a secret key $s \in \mathcal{R}_N$, satisfying $b_i + a_i s = m_i + e_i \pmod{Q}$. We define the operations as follows.

- CI-CKKS.Add and CI-CKKS.Sub: Given two ciphertexts at the same modulus and scale, compute $\mathsf{Add}(\mathsf{ct}_0, \mathsf{ct}_1) = (a_0 + a_1, b_0 + b_1)$ and $\mathsf{Sub}(\mathsf{ct}_0, \mathsf{ct}_1) = (a_0 - a_1, b_0 - b_1)$, as in standard CKKS.
- CI-CKKS.PtMult: Given a ciphertext $\mathsf{ct} = (a, b)$ and an encoded plaintext $m_1 \in \mathcal{R}_N$, compute $\mathsf{PtMult}(\mathsf{ct}, m_1) = (am_1, bm_1)$. The output scale is the product of the input scales.
- CI-CKKS.Mult: Given two ciphertexts, compute $t_0 = b_0 b_1$, $t_1 = a_0 b_1 + a_1 b_0$, and $t_2 = a_0 a_1$. The resulting ciphertext decrypts as $t_0 + t_1 s + t_2 s^2$ and is relinearized using $\mathsf{KSK}_{s^2 \to s}$, as in standard CKKS.
- CI-CKKS.KeySwitch: Given a ciphertext under s' and $\mathsf{KSK}_{s' \to s}$, apply the standard CKKS key-switching procedure over $\mathcal{R}_{N,Q}$ to obtain a ciphertext under s.
- CI-CKKS.Rescale: Given $\mathsf{ct} = (a, b)$ at modulus Q and scale Δ, compute $\mathsf{Rescale}(\mathsf{ct}, \rho) = (\lfloor a/\rho \rceil, \lfloor b/\rho \rceil)$ modulo Q/ρ, where rounding is applied coefficientwise in the basis of $\mathcal{R}_N$. The output scale is Δ/ρ.
- CI-CKKS.Rot$_k$: Given a ciphertext encrypting $\mathbf{z} \in \mathbb{R}^N$, apply the Galois automorphism $\tau_k : X \mapsto X^{5^k}$, followed by key switching using $\mathsf{KSK}_{\tau_k(s) \to s}$. This produces an encryption of $\mathsf{Rot}_k(\mathbf{z}) = (z_{(i+k) \bmod N})_{0 \le i < N}$.

Since conjugation acts trivially on $\mathcal{R}_N$, no separate conjugation operation is required.

2.3 CKKS Bootstrapping

CKKS bootstrapping refreshes a ciphertext to enable further homomorphic computations [CHK+18]. We consider the StC-first bootstrapping procedure, which consists of two linear transformations, modulus raising, and approximate modular reduction:

$$\mathsf{Boot} = \mathsf{EvalMod} \circ \mathsf{CtS} \circ \mathsf{ModRaise} \circ \mathsf{StC}.$$

The StC transformation converts the encrypted slot values into plaintext coefficients. The ModRaise operation then increases the ciphertext modulus from q_0 to a larger modulus

Q, introducing an additional term $q_0 I(X)$ into the plaintext polynomial, where $I(X)$ has integer coefficients. The CtS transformation maps the resulting coefficients back into slots, and EvalMod approximately removes the multiples of q_0 to recover the original messages. Both linear transformations can be evaluated efficiently using sparse DFT matrix factorizations [CHH18, CCS19].

Under full packing, a standard CKKS ciphertext contains $N/2$ complex slots. For real-valued inputs, StC-first bootstrapping requires only one evaluation of EvalMod. For complex-valued inputs, the real and imaginary components are processed separately, requiring two EvalMod evaluations. This also enables batched bootstrapping of two fully packed real-valued ciphertexts. Specifically, the two ciphertexts can be homomorphically combined into a single ciphertext whose real and imaginary components encode their respective messages. The resulting complex-valued ciphertext undergoes a single StC, ModRaise, and CtS procedure, followed by two EvalMod evaluations. The two refreshed real-valued ciphertexts are then recovered by separating the real and imaginary components.

2.4 Discrete CKKS

Discrete CKKS extends CKKS to homomorphic computations over encrypted discrete messages, where exact message recovery requires a sufficiently small approximation error [DMPS24, BKSS24, AKP25].

Functional bootstrapping. To evaluate lookup functions while reducing this error, we recall the CKKS functional bootstrapping method of Alexandru et al. [AKP25].

Let $f : \mathbb{Z}_p \to \mathbb{Z}_p$ be a public lookup function. We consider a CKKS ciphertext whose slots encrypt normalized messages m/p, where $m \in \mathbb{Z}_p$. As shown in Algorithm 1, functional bootstrapping first applies StC to move the messages into polynomial coefficients, followed by ModRaise and CtS. With $\Delta = q_0$, each resulting slot has the form $\Delta(I + m/p + v)$, where $I \in \mathbb{Z}$ is introduced by modulus raising and v is the normalized approximation error.

The EvalExp step computes $e^{2\pi i x}$ for $x = I + m/p + v$. It first approximates $e^{2\pi i x/2^t}$ using a Chebyshev polynomial and then applies t squarings. Since $e^{2\pi i x} = \zeta_p^m e^{2\pi i v}$, this computation removes the integer I and produces a value close to ζ_p^m.

The EvalLUT step evaluates a polynomial at the output of EvalExp to obtain $f(m)/p$. AKP constructs this polynomial using trigonometric Hermite interpolation

$$R(x) = a_0 + \sum_{k=1}^{p-1} \left(a_k \cos(2\pi k x) + b_k \sin(2\pi k x)\right),$$

satisfying $R(m/p) = f(m)/p$ and $R'(m/p) = 0$ for all $m \in \mathbb{Z}_p$. The vanishing first derivatives provide quadratic noise reduction for real input errors. For efficient evaluation, AKP writes

$$T(x) = \sum_{k=0}^{p-1} \alpha_k e^{2\pi i k x}, \qquad R(x) = \mathrm{Re}(T(x))$$

for real x. Given the output $z \approx e^{2\pi i x}$ of EvalExp, EvalLUT evaluates $\mathrm{Re}(\sum_{k=0}^{p-1} \alpha_k z^k)$ using the Paterson–Stockmeyer method [PS73].

Alexandru et al. [AKPZ26] subsequently proposed sparse trigonometric Hermite interpolation (Sparse-THI), referred to as AKPZ. They showed that AKP reduces real input noise but does not generally suppress imaginary noise arising from CKKS operations and complex exponential approximation. To address this limitation, AKPZ imposes vanishing complex derivatives up to an arbitrary order n at the interpolation points, while requiring only the real part of the interpolant to match the LUT values. This reduces both real and imaginary input errors to $O(|v|^{n+1})$, where v denotes the complex input error. By

exploiting the conjugate symmetry of Fourier coefficients for real-valued LUTs, AKPZ constructs a sparse interpolation polynomial of degree at most $(2n+1)p/2$. For fixed n, its optimized evaluation requires $O(\sqrt{p})$ homomorphic multiplications and $O(\log p)$ multiplicative levels, achieving complexity comparable to AKP.

To measure the effectiveness of noise reduction, AKPZ [AKPZ26] defines the *noise capacity* C. For an order-n Hermite interpolant, the interpolation error is modeled as $T^{-n}v^{n+1}$, where v is the input-noise magnitude and T is the noise threshold at which the interpolation error equals the input noise. Thus, the interpolation reduces noise when $v < T$. Accounting for the additional noise B introduced by homomorphic evaluation, the output noise is modeled as $v' \approx T^{-n}v^{n+1} + B$. The noise capacity C is defined as the maximum ratio v/v', attained at $v = I$:

$$I = \left(\frac{BT^n}{n}\right)^{1/(n+1)}, \qquad \log_2 C = \log_2 I - \log_2\left(\frac{n+1}{n}B\right).$$

A larger noise capacity allows more homomorphic operations between consecutive functional bootstrapping invocations.

Algorithm 1 Functional Bootstrapping for CKKS [AKP25]

Require: CKKS ciphertext ct encrypting $\mathbf{m}/p$, lookup function f
Ensure: CKKS ciphertext encrypting $f(\mathbf{m})/p$
1: $\mathsf{ct}_1 \leftarrow \mathsf{StC}(\mathsf{ct})$
2: $\mathsf{ct}_2 \leftarrow \mathsf{ModRaise}(\mathsf{ct}_1, Q_L)$
3: $\mathsf{ct}_3 \leftarrow \mathsf{CtS}(\mathsf{ct}_2)$
4: $\mathsf{ct}_4 \leftarrow \mathsf{EvalExp}(\mathsf{ct}_3)$
5: $\mathsf{ct}_5 \leftarrow \mathsf{EvalLUT}_f(\mathsf{ct}_4)$
6: **return** ct_5

3 Homomorphic DFT for Conjugate-Invariant CKKS

As described in Section 2.2, $\mathbf{V}$ maps coefficient vectors to slot vectors, and $\mathbf{V}^{-1}$ maps slot vectors to coefficient vectors. We construct sparse factorizations of these two matrices that can be evaluated directly over the conjugate-invariant ring. Starting from the standard radix-two DFT (without bit-reversal), we show that its complex intermediate values can be represented using sparse real matrices by exploiting conjugate symmetry.

3.1 Real FFT Factorization from Conjugate Symmetry

For a power of two N, let $\zeta_{4N} = \exp(2\pi i/(4N))$. We consider the DFT matrix with its columns in natural order,

$$\mathbf{U}_N = \left(\zeta_{4N}^{5^i j}\right)_{0 \le i,j < N}.$$

Relating V and U. Our real matrix $\mathbf{V}$ is closely related to this complex DFT matrix. The definition of $\mathbf{V}$ in (1) gives

$$\mathbf{V} = (\mathbf{U}_N + \overline{\mathbf{U}}_N) \cdot \mathrm{diag}(1/2, 1, \ldots, 1).$$

Then, by (2), we have

$$\mathbf{V}^{-1} = \frac{1}{2N}\left(\mathbf{U}_N^{\mathsf{T}} + \overline{\mathbf{U}}_N^{\mathsf{T}}\right).$$

Hence, for a real slot vector $\mathbf{x}$, computing $\mathbf{V}^{-1}\mathbf{x}$ reduces to computing $\mathrm{Re}(\overline{\mathbf{U}}_N^{\mathsf{T}}\mathbf{x})/N$.

3.1.1 Conjugate symmetry during real-input iFFT.

We consider the radix-two computation of $\overline{\mathbf{U}}_N^{\mathsf{T}}\mathbf{x}$ for a real input vector $\mathbf{x} = (x_0, \ldots, x_{N-1}) \in \mathbb{R}^N$. Let $\zeta = \zeta_{4N}$. For $m = 1, 2, 4, \ldots, N$, we partition the intermediate vector $\widetilde{\mathbf{z}}_m \in \mathbb{C}^N$ into m consecutive sub-vectors of length N/m. We set $\widetilde{\mathbf{z}}_1 = \mathbf{x}$. For $m = 1, 2, 4, \ldots, N/2$, $0 \le r < m$, and $0 \le j < L = N/(2m)$, the radix-two recursion is defined by

$$\begin{aligned} (\widetilde{\mathbf{z}}_{2m})_{j+rL} &= (\widetilde{\mathbf{z}}_m)_{j+2rL} + (\widetilde{\mathbf{z}}_m)_{j+(2r+1)L}, \\ (\widetilde{\mathbf{z}}_{2m})_{j+(r+m)L} &= \zeta^{-m5^j}\left((\widetilde{\mathbf{z}}_m)_{j+2rL} - (\widetilde{\mathbf{z}}_m)_{j+(2r+1)L}\right). \end{aligned} \tag{3}$$

In particular, the inputs involved at stage m have indices $j + tN/m$ for $0 \le t < m$. The following lemma gives the explicit formula of these intermediate values.

Lemma 1 (Expression of iFFT intermediate values)**.** *The recursively defined $\widetilde{\mathbf{z}}_m$ satisfies*

$$(\widetilde{\mathbf{z}}_m)_{j+r\cdot N/m} = \sum_{t=0}^{m-1} x_{j+tN/m}\zeta^{-r5^{j+tN/m}}, \qquad 0 \le j < N/m, \qquad 0 \le r < m.$$

Proof. The case $m = 1$ follows from $\widetilde{\mathbf{z}}_1 = \mathbf{x}$. Assume that the formula holds for m, and let $L = N/(2m)$. For $0 \le r < m$ and $0 \le j < L$, separating the even and odd indices in the sum for $\widetilde{\mathbf{z}}_{2m}$ with frequency index r gives

$$(\widetilde{\mathbf{z}}_{2m})_{j+rL} = (\widetilde{\mathbf{z}}_m)_{j+2rL} + (\widetilde{\mathbf{z}}_m)_{j+(2r+1)L}.$$

Since L is a power of two, $5^L \equiv 1 + 4L \pmod{8L}$, and hence $\zeta^{-m5^{j+tL}} = (-1)^t\zeta^{-m5^j}$. Thus, the sum with frequency index $r + m$ gives

$$(\widetilde{\mathbf{z}}_{2m})_{j+(r+m)L} = \zeta^{-m5^j}\left((\widetilde{\mathbf{z}}_m)_{j+2rL} - (\widetilde{\mathbf{z}}_m)_{j+(2r+1)L}\right).$$

These are exactly the two relations in (3), completing the induction. □

Using Lemma 1, we observe that when $\mathbf{x}$ is real, $\widetilde{\mathbf{z}}_m$ is almost symmetric under conjugation, namely

$$(\widetilde{\mathbf{z}}_m)_{j+(m-r)\cdot N/m} = \overline{(\widetilde{\mathbf{z}}_m)_{j+r\cdot N/m}} \cdot \zeta^{-m5^j}, \qquad 1 \le r < m.$$

We can make this conjugate symmetry perfect by compensating for the extra $-m5^j$ term over the exponent. The compensated $\mathbf{z}_m$ is defined as

$$(\mathbf{z}_m)_{j+r\cdot N/m} := (\widetilde{\mathbf{z}}_m)_{j+r\cdot N/m} \cdot \zeta^{r5^j} = \sum_{t=0}^{m-1} x_{j+tN/m}\zeta^{-r(5^{j+tN/m}-5^j)},$$

and one can verify that $\mathbf{z}_m$ has perfect conjugate symmetry

$$(\mathbf{z}_m)_{j+(m-r)\cdot N/m} = \overline{(\mathbf{z}_m)_{j+r\cdot N/m}}, \qquad 1 \le r < m.$$

This symmetry partitions $\mathbf{z}_m$ into m sub-vectors, each with length N/m

$$\mathbf{z}_m = (\mathbf{z}_{m,0}, \mathbf{z}_{m,1}, \ldots, \mathbf{z}_{m,m-1}).$$

The conjugate symmetry of $\mathbf{z}$ is summarized by the following lemma.

Lemma 2 (Conjugate symmetry of $\mathbf{z}_m$)**.** *For every power of two $m \mid N$ with $m \ge 2$, we have $\mathbf{z}_{m,m-r} = \overline{\mathbf{z}_{m,r}}$ for $1 \le r < m$. In particular, $\mathbf{z}_{m,0}$ and $\mathbf{z}_{m,m/2}$ are real, and the remaining sub-vectors occur in conjugate pairs.*

3.1.2 Constructing Real iFFT Matrices

Recurrence for the compensated values. The phase compensation also changes the radix-two recursion. Let $\omega_{m,j,r} = \zeta^{-r(5^{j+N/(2m)}-5^j)}$. Substituting the definition of $\mathbf{z}_m$ into (3), we obtain

$$\begin{aligned}(\mathbf{z}_{2m,r})_j &= (\mathbf{z}_{m,r})_j + \omega_{m,j,r}(\mathbf{z}_{m,r})_{j+N/(2m)},\\ (\mathbf{z}_{2m,r+m})_j &= (\mathbf{z}_{m,r})_j - \omega_{m,j,r}(\mathbf{z}_{m,r})_{j+N/(2m)}.\end{aligned} \qquad 0 \le r < m,\ 0 \le j < N/(2m) \tag{4}$$

For $m = 1$, this gives $(\mathbf{z}_{2,0})_j = x_j + x_{j+N/2}$ and $(\mathbf{z}_{2,1})_j = x_j - x_{j+N/2}$. At the final stage, Lemma 1 gives

$$(\widetilde{\mathbf{z}}_N)_r = \sum_{j=0}^{N-1} x_j \zeta^{-r5^j} = (\overline{\mathbf{U}}_N^{\mathsf{T}}\mathbf{x})_r, \qquad (\mathbf{z}_{N,r})_0 = \zeta^r(\widetilde{\mathbf{z}}_N)_r.$$

Consequently,

$$(\mathbf{V}^{-1}\mathbf{x})_r = \frac{1}{N}\,\mathrm{Re}\big(\zeta^{-r}(\mathbf{z}_{N,r})_0\big)\,. \tag{5}$$

Making the iFFT matrices real. Although the recursion in (4) has complex twiddle factors, we now use Lemma 2 to construct real matrices from it. By Lemma 2, we keep the real sub-vectors $\mathbf{z}_{m,0}$ and $\mathbf{z}_{m,m/2}$ and replace each conjugate pair $(\mathbf{z}_{m,r}, \mathbf{z}_{m,m-r})$ by $(\mathrm{Re}(\mathbf{z}_{m,r}), \mathrm{Im}(\mathbf{z}_{m,r}))$. The real and imaginary parts are multiplied by $\sqrt{2}$ in order to preserve the ℓ_2 norm of the pair. We also require each iFFT layer to additionally multiply the inputs by a factor of $1/\sqrt{2}$, which is required to make the iFFT matrices orthogonal. Finally, we reorder $\mathbf{z}_{m,r}$ to ensure the new iFFT matrices are diagonally sparse. An example of the first three layers is given in Figure 2.

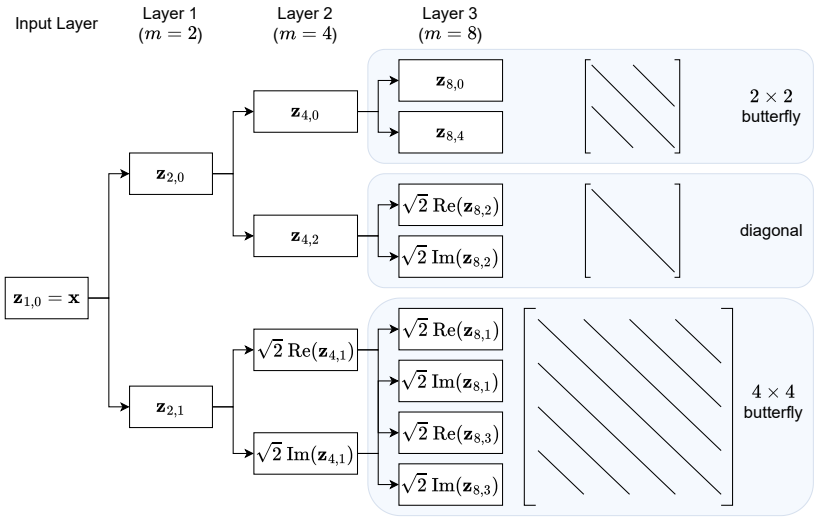


Figure 2: A three-layer example of data arrangement and sparsity patterns for butterflies during real iFFT. The extra per-layer scaling of $1/\sqrt{2}$ is not displayed.

More generally, the placement of sub-vectors in $\mathbf{z}_{2m}$ follows these rules

- **The top two**: the two real sub-vectors $\mathbf{z}_{2m,0}$ and $\mathbf{z}_{2m,m}$ obtained from $\mathbf{z}_{m,0}$.
- **Next two**: $\mathrm{Re}(\mathbf{z}_{2m,m/2})$ and $\mathrm{Im}(\mathbf{z}_{2m,m/2})$ obtained from the real sub-vector $\mathbf{z}_{m,m/2}$.

- **Remaining, four in a group**: $\mathrm{Re}(\mathbf{z}_{2m,r})$, $\mathrm{Im}(\mathbf{z}_{2m,r})$, $\mathrm{Re}(\mathbf{z}_{2m,m-r})$ and $\mathrm{Im}(\mathbf{z}_{2m,m-r})$ obtained from the real and imaginary parts of $\mathbf{z}_{m,r}$, for all $1 \le r < m/2$.

To verify that this arrangement is well-defined, we need to check if it remains valid after moving to the next iFFT layer, and if the output values in each size-2 or size-4 group are indeed linear combinations of the source values in the same group. The first point, that the top two sub-vectors are always $\mathbf{z}_{2m,0}$ and $\mathbf{z}_{2m,m}$, can be proven by induction on m. The second point will be proved together with the next lemma.

Lemma 3 (Decomposition of iFFT into real sparse matrices)**.** *Let $L = N/(2m)$. Define two N/m-sized matrices $\mathbf{G}_{m,real}$ and $\mathbf{G}_{m,split}$ by*

$$\mathbf{G}_{m,real}\,\mathbf{z}_{m,0} = \frac{1}{\sqrt{2}}\begin{bmatrix}\mathbf{z}_{2m,0}\\ \mathbf{z}_{2m,m}\end{bmatrix}, \qquad \mathbf{G}_{m,split}\,\mathbf{z}_{m,m/2} = \begin{bmatrix}Re(\mathbf{z}_{2m,m/2})\\ Im(\mathbf{z}_{2m,m/2})\end{bmatrix}.$$

The former has three nonzero diagonals at $\{0, \pm L\}$, while the latter is a diagonal matrix.

For $1 \le r < m/2$, the $2N/m$-sized matrix $\mathbf{G}_{m,r}$ defined by

$$\mathbf{G}_{m,r}\begin{bmatrix}Re(\mathbf{z}_{m,r})\\ Im(\mathbf{z}_{m,r})\end{bmatrix} = \frac{1}{\sqrt{2}}\begin{bmatrix}Re(\mathbf{z}_{2m,r})\\ Im(\mathbf{z}_{2m,r})\\ Re(\mathbf{z}_{2m,m-r})\\ Im(\mathbf{z}_{2m,m-r})\end{bmatrix}$$

has no more than seven nonzero diagonals at offset $\{0, \pm L, \pm 2L, \pm 3L\}$.

All $\mathbf{G}_{m,real}$, $\mathbf{G}_{m,split}$, and $\mathbf{G}_{m,r}$ are real and orthogonal.

Proof. The case of $\mathbf{z}_{m,0}$ corresponds to the case of $r = 0$ in (4), giving $\omega_{m,j,r} = 1$ and a real orthogonal $\mathbf{G}_{m,\mathrm{real}}$ with three nonzero diagonals

$$\mathbf{G}_{m,\mathrm{real}} = \frac{1}{\sqrt{2}}\begin{bmatrix}\mathbf{I}_L & \mathbf{I}_L\\ \mathbf{I}_L & -\mathbf{I}_L\end{bmatrix}.$$

The case of $\mathbf{z}_{m,m/2}$ corresponds to $r = m/2$, in which case $\omega_{m,j,r} \in \{\pm \mathrm{i}\}$. Separating of the real and imaginary parts transforms the recurrence relationship into

$$\begin{aligned}\mathrm{Re}(\mathbf{z}_{2m,m/2})_j &= (\mathbf{z}_{m,m/2})_j\\ \mathrm{Im}(\mathbf{z}_{2m,m/2})_j &= \mathrm{Im}(\omega_{m,j,r})(\mathbf{z}_{m,m/2})_{j+L}.\end{aligned}$$

Then $\mathbf{G}_{m,\mathrm{split}}$ is a diagonal matrix with ± 1 on its diagonal.

For $\mathbf{G}_{m,r}$, we look at the size-4 butterfly for some fixed r and j. Let $\omega_{m,j,r} = \alpha + i\beta$, the recurrence relationships are

$$\begin{aligned}\mathrm{Re}(\mathbf{z}_{2m,r})_j &= &&\mathrm{Re}(\mathbf{z}_{m,r})_j + \alpha\mathrm{Re}(\mathbf{z}_{m,r})_{j+L} - \beta\mathrm{Im}(\mathbf{z}_{m,r})_{j+L}\\ \mathrm{Im}(\mathbf{z}_{2m,r})_j &= &&\mathrm{Im}(\mathbf{z}_{m,r})_j + \beta\mathrm{Re}(\mathbf{z}_{m,r})_{j+L} + \alpha\mathrm{Im}(\mathbf{z}_{m,r})_{j+L}\\ \mathrm{Re}(\mathbf{z}_{2m,m-r})_j &= \mathrm{Re}(\mathbf{z}_{2m,m+r})_j = &&\mathrm{Re}(\mathbf{z}_{m,r})_j - \alpha\mathrm{Re}(\mathbf{z}_{m,r})_{j+L} + \beta\mathrm{Im}(\mathbf{z}_{m,r})_{j+L}\\ \mathrm{Im}(\mathbf{z}_{2m,m-r})_j &= -\mathrm{Im}(\mathbf{z}_{2m,m+r})_j = &&-\mathrm{Im}(\mathbf{z}_{m,r})_j + \beta\mathrm{Re}(\mathbf{z}_{m,r})_{j+L} + \alpha\mathrm{Im}(\mathbf{z}_{m,r})_{j+L}.\end{aligned}$$

This gives the expression of the 4×4 sub-matrix in $\mathbf{G}_{m,r}$ with indices $\{j, j+L, j+2L, j+3L\}$, omitting the per-level scaling of $1/\sqrt{2}$.

$$\begin{bmatrix}\mathrm{Re}(\mathbf{z}_{2m,r})_j\\ \mathrm{Im}(\mathbf{z}_{2m,r})_j\\ \mathrm{Re}(\mathbf{z}_{2m,m-r})_j\\ \mathrm{Im}(\mathbf{z}_{2m,m-r})_j\end{bmatrix} = \begin{bmatrix}1 & \alpha & 0 & -\beta\\ 0 & \beta & 1 & \alpha\\ 1 & -\alpha & 0 & \beta\\ 0 & \beta & -1 & \alpha\end{bmatrix}\begin{bmatrix}\mathrm{Re}(\mathbf{z}_{m,r})_j\\ \mathrm{Re}(\mathbf{z}_{m,r})_{j+L}\\ \mathrm{Im}(\mathbf{z}_{m,r})_j\\ \mathrm{Im}(\mathbf{z}_{m,r})_{j+L}\end{bmatrix},$$

whose normalization by $1/\sqrt{2}$ is orthogonal because $\alpha^2 + \beta^2 = 1$. Enumerating over all choices of r and j gives the full $\mathbf{G}_{m,r}$. It is real and has at most seven nonzero diagonals at $\{0, \pm L, \pm 2L, \pm 3L\}$, as desired.

□

The first real iFFT matrix is

$$\mathbf{F}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} \mathbf{I}_{N/2} & \mathbf{I}_{N/2} \\ \mathbf{I}_{N/2} & -\mathbf{I}_{N/2} \end{pmatrix}.$$

For $m = 2, 4, \ldots, N/2$, the $(\log_2 m + 1)$-st iFFT layer is block diagonal, with $\mathbf{G}_{m,\text{real}}$, $\mathbf{G}_{m,\text{split}}$, and $\mathbf{G}_{m,r}$ on its diagonal before the final phase correction.

The last layer with $m = N$, however, needs to additionally remove the phase factor ζ^r from $(\mathbf{z}_{N,r})_0$, according to (5). Before this phase removal, the slot vector begins with two real numbers[2] $\mathbf{z}_{N,0}$ and $\mathbf{z}_{N,N/2}$, followed by the real and imaginary pairs of each conjugate pair with $1 \le r < N/2$. The phase removal leaves $\mathbf{z}_{N,0}$ untouched and multiplies $\mathbf{z}_{N,N/2}$ by $\mathrm{Re}(\zeta^{N/2})$. This scaling is absorbed into $\mathbf{S}$, rather than $\mathbf{F}_\ell$. For $1 \le r < N/2$, let $\theta_r = \pi r/(2N)$. The phase removal applies a 2×2 butterfly to each real/imaginary pair

$$\begin{pmatrix} \mathrm{Re}(\zeta^{-r}\mathbf{z}_{N,r}) \\ -\mathrm{Im}(\zeta^{-r}\mathbf{z}_{N,r}) \end{pmatrix} = \begin{pmatrix} \cos\theta_r & \sin\theta_r \\ \sin\theta_r & -\cos\theta_r \end{pmatrix} \begin{pmatrix} \mathrm{Re}(\mathbf{z}_{N,r}) \\ \mathrm{Im}(\mathbf{z}_{N,r}) \end{pmatrix}.$$

This transformation is again real and orthogonal. The transformation corresponds to a block-diagonal matrix with two 1×1 blocks followed by many 2×2 blocks. Therefore, merging it with the block-diagonal matrix formed by $G_{N/2,\text{real}}$, $G_{N/2,\text{split}}$, and $G_{N/2,r}$ preserves the block structure and gives at most seven nonzero cyclic diagonals in the last layer.

The overall iFFT factorization. We denote the iFFT matrices obtained in the above manner as $\mathbf{F}_1, \ldots, \mathbf{F}_{\log_2 N}$. The full real iFFT is given by the following theorem.

Theorem 1 (Real DFT matrix decomposition)**.** *Let* $\mathbf{S} = \mathrm{diag}(1/\sqrt{N}, 1/\sqrt{2N}, \ldots, 1/\sqrt{2N})$ *and* $\ell = \log_2 N$*. Then the permuted real iFFT factorizes into*

$$\mathbf{P}\mathbf{V}^{-1} = \mathbf{S}\mathbf{F}_\ell \cdots \mathbf{F}_1,$$

where $\mathbf{P}$ *is the permutation determined by the ordering of the final real coordinates (not a bit-reversal permutation). Each* $\mathbf{F}_i$ *is a real orthogonal matrix with at most seven nonzero cyclic diagonals.*

Since $\mathbf{F}_i^{-1} = \mathbf{F}_i^\mathsf{T}$*, the reverse transformation is given by*

$$\mathbf{V}\mathbf{P}^\mathsf{T} = \mathbf{F}_1^\mathsf{T} \cdots \mathbf{F}_\ell^\mathsf{T} \mathbf{S}^{-1}.$$

Similar to standard CKKS bootstrapping, the permutation $\mathbf{P}$ is not evaluated separately, because EvalMod is insensitive to the order of slots and the two permutations in CtS and StC cancel.

3.2 Homomorphic Evaluation

We evaluate the real matrices using the diagonal method recalled in Section 2.1. For the forward transformation, the diagonal scaling $\mathbf{S}$ is included in the plaintext diagonal vectors of the last matrix. Since $\mathbf{S}$ is diagonal, this does not change its nonzero diagonal indices.

Lemma 4 (Nonzero diagonals)**.** *For* $1 \le i < \ell$*, let* $h = N/2^i$*. The nonzero diagonal indices of* $\mathbf{F}_i$ *are contained in*

$$\{-2h, -h, 0, h, 2h, 3h\} \pmod N.$$

The first matrix has at most two nonzero diagonals, the second has at most three, and every remaining matrix before the last has at most six. After the final 2×2 *matrices are included,* $\mathbf{F}_\ell$ *has at most seven nonzero diagonals.*

[2]Each sub-vector has length 1 in this case, so we abuse the notation $\mathbf{z}_{N,r}$ for $(\mathbf{z}_{N,r})_0$.

Proof. For the real subvector with local index 0, the two input positions differ by h, giving the offsets 0 and $\pm h$. The subvector with local index $m/2$ does not mix different positions. For every nonreal pair, the corresponding 4×4 matrix acts on four consecutive subvectors of length h, and its nonzero entries give the offsets $-2h, -h, 0, h, 2h$, and $3h$. For the first matrix, $h = N/2$, so $h \equiv -h \pmod N$. The second matrix has no 4×4 case. The final 2×2 matrices give at most seven nonzero diagonals in $\mathbf{F}_\ell$. □

These matrices can be homomorphically evaluated via the standard algorithm for linear transformations.

For $N \geq 8$, the numbers of nonzero diagonals are bounded by $2, 3, 6, \ldots, 6, 7$. Therefore, a naive layer-by-layer implementation uses at most $6\ell - 6$ PtMult operations and $5\ell - 6$ nontrivial Rot operations, with depth ℓ.

The reverse transformation is evaluated in the same way using the transposed matrices in reverse order, with $\mathbf{S}^{-1}$ included in the plaintext diagonal vectors of the first evaluated matrix. Transposition changes the diagonal indices only by sign, so the same operation counts apply.

3.2.1 Applying Level Collapsing

The radix-two method evaluates one sparse matrix at each level and therefore has depth $\ell = \log_2 N$. Following [CHH18, CCS19], we reduce the depth using level collapsing. For a power of two r, we multiply $\log_2 r$ consecutive matrices during plaintext preprocessing and evaluate their product as one homomorphic linear transformation. The number of homomorphic matrix evaluations is therefore reduced from $\log_2 N$ to $\log_r N$.

To bound the cost of one collapsed matrix, recall that the diagonal vectors of a matrix product satisfy

$$\mathrm{diag}_k(\mathbf{AB}) = \sum_{a+b \equiv k \pmod N} \mathrm{diag}_a(\mathbf{A}) \odot \mathsf{Rot}_a(\mathrm{diag}_b(\mathbf{B})) .$$

Hence a nonzero diagonal index of a product is obtained by adding diagonal indices of its factors.

Lemma 5 (Collapsed matrices)**.** *Let r be a power of two and consider the product of $\log_2 r$ consecutive matrices. If h is the smallest diagonal spacing among these matrices, then the nonzero diagonal indices of the product are contained in an arithmetic progression of $O(r)$ multiples of h. Hence the collapsed matrix has $O(r)$ nonzero diagonal vectors.*

Proof. By Lemma 4, the diagonal indices of each matrix are constant multiples of its spacing. For $\log_2 r$ consecutive matrices, these spacings are $h, 2h, \ldots, (r/2)h$. Since diagonal indices add under matrix multiplication, every diagonal index of their product is a multiple of h. Moreover, the sum of the absolute values of the possible offsets is $O(r)h$. Thus all nonzero diagonal indices lie among $O(r)$ consecutive multiples of h. □

There are $\log_r N$ collapsed matrices. Using the diagonal method directly, the transform therefore requires $O(r \log_r N)$ PtMult operations and rotations, with depth $\log_r N$. Thus level collapsing reduces the depth at the cost of more operations per matrix evaluation. If $\log_2 r$ does not divide ℓ, the last product uses fewer matrices.

3.2.2 Applying BSGS

Lemma 5 shows that the nonzero diagonal indices of each collapsed matrix are contained in an arithmetic progression. We therefore apply the baby-step giant-step method to each collapsed matrix, following [CHH18, CCS19].

A collapsed matrix with $O(r)$ nonzero diagonal vectors can be evaluated using $O(\sqrt{r})$ rotations and $O(r)$ PtMult operations by BSGS [HS18]. Applying BSGS independently to all $\log_r N$ collapsed matrices gives the following result.

Corollary 1 (Homomorphic DFT)**.** *For a power of two r, the conjugate-invariant DFT and inverse DFT can each be evaluated using $O(\sqrt{r}\log_r N)$ rotations and $O(r\log_r N)$ PtMult operations, with depth $\log_r N$.*

Level collapsing gives a trade-off between depth and the number of plaintext–ciphertext multiplications, while BSGS reduces the number of rotations.

4 Bootstrapping over the Conjugate-Invariant Ring

We now apply the linear transformations of Section 3 to CKKS bootstrapping. The output of our coefficients-to-slots transformation is a permuted version of the coefficient vector. As in previous works on CKKS bootstrapping [CHH18, CCS19], we do not reorder the coefficients by evaluating the inverse permutation homomorphically. Instead, we keep the permuted order between the two linear transformations. We then extend the native bootstrapping procedure to functional bootstrapping over real slots.

4.1 Previous Methods

Previous approaches bootstrap CI-CKKS ciphertexts through standard CKKS by converting the ciphertexts before and after bootstrapping.

Lattigo [Tun26]. Lattigo converts a CI ciphertext of rank N to a standard CKKS ciphertext of rank $2N$. Its RealToComplex operation expands the compressed CI polynomial into its conjugate-symmetric representation in the larger ring and then key-switches the ciphertext to a standard CKKS secret key. The resulting ciphertext encrypts N real-valued complex slots. Two CI ciphertexts encrypting $\mathbf{u}, \mathbf{v} \in \mathbb{R}^N$ are combined into a single standard CKKS ciphertext encrypting $\mathbf{u} + i\mathbf{v}$. After standard CKKS bootstrapping, ComplexToReal switches the result back to the CI secret, adds the conjugate polynomial to extract its real part, and compresses the coefficients into the CI ring. Applying the same procedure after multiplication by $-i$ recovers $\mathbf{v}$. The conversions require key switching but no additional multiplicative levels. For full-slot bootstrapping, the real and imaginary outputs of CtS undergo separate EvalMod evaluations in parallel.

Bae et al. [BCKK25]. Bae et al. convert a CI ciphertext with N real slots into a standard CKKS ciphertext with $N/2$ complex slots. The conversion, denoted by FromCI, maps

$$(v_j)_{0\le j<N} \longmapsto (v_j + iv_{j+N/2})_{0\le j<N/2}.$$

Unlike Lattigo, this method uses both the real and imaginary components of each complex slot without doubling the ring rank. The reverse conversion, denoted by ToCI, separates the real and imaginary components and restores the original N real slots. These conversions require masking multiplications to select the corresponding components, with one additional multiplicative level consumed in either direction of conversion. They also propose to replace the two CKKS EvalMod with one CI-CKKS EvalMod in a full-slot bootstrapping to improve the bootstrapping efficiency.

Kim et al. [KLN+26]. Kim et al. investigate how to combine the above CI-to-standard conversion FromCI with standard CKKS StC. They also consider fusing an additional linear transformation required by AES evaluation into this conversion. The fused computation preserves the number of rotations and eliminates the two additional multiplicative levels required by the AES-related map and FromCI. However, it operates over a standard CKKS ring of rank $2N$, approximately doubling the cost of StC. The reverse conversion ToCI still consumes one multiplicative level.

4.2 Bootstrapping with Real Linear Transformations

Using the real linear transformations from Section 3, we construct bootstrapping directly over $\mathcal{R}_N$, avoiding the ring dimension doubling required by the standard CKKS-based approach. Let $\mathbf{x} \in \mathbb{R}^N$ denote the input slot vector. Ignoring approximation and rounding errors, the procedure is

$$\begin{array}{rccc} \text{Slots:} & \mathbf{x} & \mathbf{x} + \frac{q_0}{\Delta}\mathbf{PI} & \xrightarrow{\mathsf{EvalMod}} \quad \mathbf{x} \\ & \downarrow \mathsf{StC} & \uparrow \mathsf{CtS} & \\ \text{Coefficients:} & \Delta\mathbf{P}^{\mathsf{T}}\mathbf{x} & \xrightarrow{\mathsf{ModRaise}} \quad \Delta\mathbf{P}^{\mathsf{T}}\mathbf{x} + q_0\mathbf{I} & \end{array}$$

where Δ is the plaintext scale before modulus raising and $\mathbf{I} \in \mathbb{Z}^N$ is the integer overflow introduced by modulus raising. Since $\mathbf{P}$ is a permutation matrix, $\mathbf{PI}$ remains an integer vector. Thus, EvalMod removes the multiples of q_0/Δ and recovers the original slot values.

By Corollary 1, the two linear transformations require $O(\sqrt{r}\log_r N)$ rotations and $O(r\log_r N)$ plaintext–ciphertext multiplications in total, with multiplicative depth $2\log_r N$. Unlike bootstrapping via standard CKKS, all operations are performed directly over the rank-N conjugate-invariant ring, without ring switching.

4.3 Functional Bootstrapping

We now consider functional bootstrapping for discrete CKKS over the CIR, where discrete messages $m \in \mathbb{Z}_p$ are encoded in real-valued slots. Our construction follows the general framework of AKP [AKP25], as described in Section 2.4. Specifically, we adopt the StC-first procedure and implement StC and CtS using the real DFT factorizations developed in Section 3. The EvalExp step is implemented using Chebyshev approximation followed by repeated double-angle steps as before. The remaining challenge is to construct an efficient Hermite interpolation method for evaluating arbitrary lookup functions directly over the CIR.

Let $p \geq 2$ be a power of two and let $f : \mathbb{Z}_p \to \mathbb{Z}_p$ be a public lookup function. After ModRaise and CtS, each slot contains a value of the form $x = I + m/p + v$, where $m \in \mathbb{Z}_p$, $I \in \mathbb{Z}$ is an integer introduced by modulus raising, and $v \in \mathbb{R}$ denotes the approximation error. Our goal is to evaluate $f(m)/p$ while removing the integer term I and providing noise cleaning to reduce the input error v. To this end, we construct a real-valued, 1-periodic Hermite interpolant whose derivatives vanish at the interpolation points, thereby suppressing the input error to higher order.

A straightforward approach is to emulate complex CKKS arithmetic using two CI-CKKS ciphertexts, which separately store the real and imaginary parts of each complex slot. Complex arithmetic can then be performed using real ciphertext operations. For example, complex multiplication can be computed as

$$(a + ib)(d + ie) = (ad - be) + i\big((a + b)(d + e) - ad - be\big),$$

requiring three real ciphertext multiplications. Thus, existing interpolation methods, including AKP and AKPZ, can be implemented over the CI-CKKS.

We also present two interpolation methods for real-valued ciphertexts. The first adapts the real-valued Fourier–Hermite interpolant of AKP to CI-CKKS and introduces a correction to reduce the numerical instability identified by AKPZ. The second constructs a univariate Hermite interpolant using a single cosine. We then describe the homomorphic evaluation of both methods and analyze their computational complexity.

4.3.1 Interpolation Methods over the Conjugate-Invariant Ring

For an interpolation order $n \geq 1$, our goal is to construct a real-valued, 1-periodic function R satisfying

$$R(m/p) = \frac{f(m)}{p}, \qquad R^{(j)}(m/p) = 0, \quad 1 \leq j \leq n,$$

for every $m \in \mathbb{Z}_p$. Periodicity removes the integer term introduced by modulus raising, while the vanishing derivatives provide $(n+1)$-th-order noise cleaning:

$$R(I + m/p + v) = \frac{f(m)}{p} + O(|v|^{n+1})$$

for sufficiently small real v under exact evaluation.

Real evaluation of the AKP interpolant. We first consider the original real-valued Fourier–Hermite interpolant of AKP [AKP25]. As shown in Appendix A of AKPZ [AKPZ26], this interpolant can be rewritten using Chebyshev identities as

$$R(x) = A(\cos(2\pi x)) + \sin(2\pi x)B(\cos(2\pi x)),$$

where A and B are real polynomials determined by the Fourier coefficients. To evaluate this expression homomorphically, we can first approximate the cosine and sine functions in two separate CI-CKKS ciphertexts using Chebyshev polynomials followed by repeated double-angle steps and then evaluate A and B. For convenience, let $c \approx \cos(2\pi x) = \cos(2\pi(m/p + v))$ and $s \approx \sin(2\pi x) = \sin(2\pi(m/p + v))$ denote the homomorphically computed approximations, where the integer term I is removed by periodicity.

However, independently approximating cosine and sine can introduce numerical instability as observed in Appendix A of AKPZ [AKPZ26]. Due to approximation errors, the computed values c and s generally satisfy $c^2 + s^2 = 1 + \varepsilon'$ rather than 1, where ε' denotes a small off-circle error.

To see its effect, let $c_0 = \cos(2\pi x)$ and $s_0 = \sin(2\pi x)$ denote the exact values at an interpolation point. For a radial perturbation $(c, s) = \sqrt{1+\varepsilon'}(c_0, s_0)$, a first-order Taylor expansion gives

$$A(c) + sB(c) = R(x) + \frac{\varepsilon'}{2}\big(c_0 A'(c_0) + s_0 B(c_0) + c_0 s_0 B'(c_0)\big) + O((\varepsilon')^2).$$

Since the order-n Hermite conditions imply $R(x) = f(m)/p + O(|v|^{n+1})$, the output error is $O(|v|^{n+1}) + O(|\varepsilon'|)$. Consequently, the output may contain an additional $O(|\varepsilon'|)$ error, which can dominate the expected $O(|v|^{n+1})$ noise reduction.

We address this issue by incorporating a correction into the final double-angle step. Let $c' \approx \cos(\pi x)$ and $s' \approx \sin(\pi x)$ denote the approximate values before this step, and write $c'^2 + s'^2 = 1 + \varepsilon$. The ordinary double-angle formulas give $c = c'^2 - s'^2$ and $s = 2c's'$, satisfying $c^2 + s^2 = (1+\varepsilon)^2 = 1 + \varepsilon'$, where $\varepsilon' = 2\varepsilon + \varepsilon^2$. Thus, $\sqrt{c^2 + s^2} = 1 + \varepsilon$. To reduce this deviation, we approximate $1/(1+\varepsilon)$ by its first-order Taylor expansion $1 - \varepsilon$ and compute

$$C = c(1 - \varepsilon), \qquad S = s(1 - \varepsilon).$$

For higher-order correction, we replace $1-\varepsilon$ with $\sum_{j=0}^{n}(-\varepsilon)^j$. Since $(1+\varepsilon)\sum_{j=0}^{n}(-\varepsilon)^j = 1 - (-\varepsilon)^{n+1}$, the radial error is reduced to $O(|\varepsilon|^{n+1})$.

Theorem 2. *Let $R(x) = A(\cos(2\pi x)) + \sin(2\pi x)B(\cos(2\pi x))$ be an order-n Hermite interpolant. For $x = I + m/p + v$, suppose $(c, s) = (1+\varepsilon)(\cos(2\pi x), \sin(2\pi x))$, where $|\varepsilon| < 1$. Define the corrected values as*

$$C = c\sum_{j=0}^{n}(-\varepsilon)^j, \qquad S = s\sum_{j=0}^{n}(-\varepsilon)^j. \tag{6}$$

The homomorphic evaluation of $A(C) + SB(C)$ yields

$$\frac{f(m)}{p} + O(|v|^{n+1}) + (-1)^n D_m \varepsilon^{n+1} + O((|v| + |\varepsilon|)^{n+2}) + e_{\text{eval}}.$$

where e_{eval} denotes the homomorphic evaluation error and

$$D_m = c_m A'(c_m) + s_m B(c_m) + c_m s_m B'(c_m),$$

with $c_m = \cos(2\pi m/p)$ and $s_m = \sin(2\pi m/p)$.

Proof. Since $(1+\varepsilon)\sum_{j=0}^{n}(-\varepsilon)^j = 1 - (-\varepsilon)^{n+1}$, the corrected values satisfy $(C, S) = (1 - (-\varepsilon)^{n+1})(\cos(2\pi x), \sin(2\pi x))$. Taylor expansion gives $A(C) + SB(C) = R(x) + (-1)^n D_m \varepsilon^{n+1} + O(|v||\varepsilon|^{n+1} + |\varepsilon|^{2n+2})$. By the Hermite interpolation conditions [AKP25], we have $R(x) = f(m)/p + O(|v|^{n+1})$. Combining these expressions with the evaluation error proves the claim. □

Chebyshev-node Hermite interpolation. We construct the interpolant as a real polynomial in a single cosine variable. We choose $u(x) = \cos(2\pi x + \pi/(2p))$, which maps the p messages to the distinct roots of the Chebyshev polynomial T_p. For an interpolation order $n \geq 1$, we construct a real polynomial H and define

$$R(x) = H(u(x)) = a_0 + \sum_{k=1}^{(n+1)p-1} a_k \cos\left(2\pi k x + \frac{k\pi}{2p}\right), \tag{7}$$

where $H(z) = \sum_{k=0}^{(n+1)p-1} a_k T_k(z)$ and $T_k(\cos\theta) = \cos(k\theta)$.

Let $u_m = u(m/p)$ for $m \in \mathbb{Z}_p$. We impose $H(u_m) = f(m)/p$ and $H^{(j)}(u_m) = 0$ for $1 \leq j \leq n$. These conditions ensure $R(m/p + v) = f(m)/p + O(|v|^{n+1})$, providing higher-order noise suppression. The following theorem gives the coefficients of H and the corresponding error bound.

Theorem 3. *For any $f : \mathbb{Z}_p \to \mathbb{Z}_p$ and $n \geq 1$, there exists a unique real polynomial H of degree at most $(n+1)p - 1$ satisfying the above interpolation conditions. Its Chebyshev coefficients are*

$$a_0 = \frac{1}{p^2}\sum_{m=0}^{p-1} f(m),$$

and

$$a_{|k+2pq|} = \frac{2(-1)^q}{p^2}\sum_{m=0}^{p-1} f(m)T_k(u_m) \prod_{\substack{r=-\lceil n/2\rceil \\ r\neq q}}^{\lfloor n/2\rfloor} \frac{k+2pr}{2p(r-q)}, \tag{8}$$

for $1 \leq k < p$ and $-\lceil n/2\rceil \leq q \leq \lfloor n/2\rfloor$. All remaining coefficients are zero. Moreover, the interpolant $R(x) = H(u(x))$ satisfies

$$\left| R\left(\frac{m}{p} + v\right) - \frac{f(m)}{p} \right| \leq K_{f,n}|v|^{n+1} + O(|v|^{n+2}), \tag{9}$$

as $v \to 0$*, where*

$$K_{f,n} = \frac{(2\pi)^{n+1}}{(n+1)!} \max_{m \in \mathbb{Z}_p} \left| \sum_{k=1}^{(n+1)p-1} k^{n+1} a_k \cos\left(\frac{2\pi k m}{p} + \frac{k\pi}{2p} + \frac{(n+1)\pi}{2} \right) \right|. \tag{10}$$

Proof. Let $\theta_m = 2\pi m/p + \pi/(2p)$. For each $1 \le k < p$, define $t_q = k + 2pq$ and let w_q denote the product in (8). These are the Lagrange interpolation weights at zero for the nodes t_q, and hence satisfy $\sum_q w_q t_q^j = \delta_{j0}$ for $0 \le j \le n$. Since $2p\theta_m = (4m+1)\pi$, the j-th derivative of $\cos(t_q\theta)$ at θ_m equals $(-1)^q t_q^j \cos(k\theta_m + j\pi/2)$. By (8), we have $a_{|t_q|} = (-1)^q w_q (2/p^2) \sum_{\ell=0}^{p-1} f(\ell) T_k(u_\ell)$. Thus, the Lagrange weight identities ensure that the first n derivatives of $H(\cos\theta)$ vanish at every θ_m. Discrete Chebyshev orthogonality gives $H(u_m) = f(m)/p$. Since $\sin\theta_m \neq 0$, the chain rule implies $H^{(j)}(u_m) = 0$ for $1 \le j \le n$. Uniqueness follows because a nonzero polynomial of degree at most $(n+1)p-1$ cannot have p distinct roots of multiplicity $n+1$. Finally, differentiating (7) gives the expression for $K_{f,n}$ in (10), and Taylor expansion yields (9). □

Remark 1. The minimum spacing between the Chebyshev nodes is $2\sin(\pi/p)\sin(\pi/(2p)) = \Theta(p^{-2})$. This clustering increases the precision required for evaluating the cosine, since additive errors can become comparable to the spacing between adjacent nodes. However, the cosine derivative is also $O(p^{-1})$ near the endpoints, which reduces the effect of phase errors. The noise coefficient $K_{f,n}$ depends on the lookup function and can grow as $\Theta(p^{n+1})$.

4.3.2 Homomorphic Evaluation

Real evaluation for the AKP interpolant. We evaluate the AKP interpolant using two CI-CKKS ciphertexts to approximate $\cos(2\pi x)$ and $\sin(2\pi x)$. Specifically, we first approximate $\cos(2\pi x/2^t)$ and $\sin(2\pi x/2^t)$ using Chebyshev polynomials, followed by t double-angle steps. Let $\mathsf{ct}_{c'}$ and $\mathsf{ct}_{s'}$ denote the ciphertexts encrypting $c' \approx \cos(\pi x)$ and $s' \approx \sin(\pi x)$ immediately before the final double-angle step.

We incorporate the radial correction into this final step. The double-angle formulas compute $c = c'^2 - s'^2$ and $s = 2c's'$, while the radial error $\varepsilon = c'^2 + s'^2 - 1$ can be obtained by reusing the two ciphertext squares without additional multiplications.

For a target correction order $n \ge 1$, we set $r = \lceil \log_2(n+1) \rceil$. To evaluate the correction in (6), we use the factorization

$$\sum_{j=0}^{2^r-1} (-\varepsilon)^j = (1-\varepsilon) \prod_{j=1}^{r-1} (1 + \varepsilon^{2^j}).$$

This allows us to achieve at least the target correction order by computing successive powers of ε through repeated squaring and applying the corresponding factors to both c and s. The complete procedure is given in Algorithm 2.

As shown in Algorithm 2, the final double-angle step already computes c'^2 and s'^2, allowing us to obtain $\varepsilon = c'^2 + s'^2 - 1$ without additional ciphertext multiplications. For $r = \lceil \log_2(n+1) \rceil$, our correction requires $3r-1$ additional ciphertext multiplications and r multiplicative levels, reducing the radial error to $O(|\varepsilon|^{2^r})$. The polynomial evaluations of A and B remain unchanged.

For comparison, AKPnorm [AKPZ26] applies a Newton iteration to approximate $1/\sqrt{c^2+s^2}$, computing $C = c(3-c^2-s^2)/2$ and $S = s(3-c^2-s^2)/2$. Over CI-CKKS, this requires four real ciphertext multiplications and two additional multiplicative levels. Reusing the intermediate squares reduces the multiplication count to three, but not the depth. In contrast, our first-order correction requires only two multiplications and one level. For higher orders, repeating the Newton iteration r times requires $4r-1$ multiplications and $2r$ levels with the initial squares reused, compared with $3r-1$ multiplications and r levels for our method.

Algorithm 2 Real Evaluation of the AKP Interpolant

Require: CI-CKKS ciphertexts $\mathsf{ct}_{c'}$, $\mathsf{ct}_{s'}$; correction order $n \geq 1$
Ensure: Ciphertext encrypting the corrected interpolant value

$r \leftarrow \lceil \log_2(n+1) \rceil$
$\mathsf{ct}_{c'^2} \leftarrow \mathsf{ct}_{c'}^2, \quad \mathsf{ct}_{s'^2} \leftarrow \mathsf{ct}_{s'}^2$
$\mathsf{ct}_c \leftarrow \mathsf{ct}_{c'^2} - \mathsf{ct}_{s'^2}$
$\mathsf{ct}_s \leftarrow 2\mathsf{ct}_{c'}\mathsf{ct}_{s'}$
$\mathsf{ct}_\varepsilon \leftarrow \mathsf{ct}_{c'^2} + \mathsf{ct}_{s'^2} - 1$
$\mathsf{ct}_C \leftarrow \mathsf{ct}_c(1 - \mathsf{ct}_\varepsilon), \quad \mathsf{ct}_S \leftarrow \mathsf{ct}_s(1 - \mathsf{ct}_\varepsilon)$
for $j = 1, \ldots, r-1$ **do**
    $\mathsf{ct}_\varepsilon \leftarrow \mathsf{ct}_\varepsilon^2$
    $\mathsf{ct}_C \leftarrow \mathsf{ct}_C(1 + \mathsf{ct}_\varepsilon)$
    $\mathsf{ct}_S \leftarrow \mathsf{ct}_S(1 + \mathsf{ct}_\varepsilon)$
end for
return $A(\mathsf{ct}_C) + \mathsf{ct}_S B(\mathsf{ct}_C)$

Single-cosine Hermite interpolation. For single-cosine Hermite interpolation, we evaluate $H(u(x))$, where $u(x) = \cos(2\pi x + \pi/(2p))$. Following AKP [AKP25], we approximate the cosine using a degree-d Chebyshev polynomial followed by t double-angle steps. We then evaluate H using the Paterson–Stockmeyer method [PS73], with its coefficients precomputed from the public lookup function.

The cosine computation requires $O(\sqrt{d} + t)$ ciphertext multiplications. Since H has degree at most $(n+1)p-1$, its evaluation requires $O(\sqrt{(n+1)p})$ ciphertext multiplications. The total multiplicative depth is bounded by

$$\lceil \log_2 d \rceil + t + \lceil \log_2((n+1)p - 1) \rceil + O(1).$$

For fixed n, d, and t, the evaluation requires $O(\sqrt{p})$ ciphertext multiplications and $O(\log p)$ multiplicative depth. Unlike the evaluation of the AKP interpolant, this method requires only one real-valued ciphertext throughout the polynomial evaluation. Neither method requires ciphertext rotations.

5 Implementation

We implemented our bootstrapping and functional bootstrapping methods in Go using Lattigo 6.2.0 [Tun26]. All experiments are performed on an Intel Xeon Gold 6248R processor running Linux/amd64 with Go 1.24. We use a single CPU core and report the average time of ten evaluations after one warm-up.

5.1 Parameter Selection

We use the conjugate-invariant ring of dimension $N = 2^{16}$, which supports N real slots. For functional bootstrapping, we implement three methods over CI-CKKS: the real-valued AKP interpolant with correction, our single-cosine Hermite interpolation, and Sparse-THI with complex arithmetic simulated using real ciphertexts. We compare these methods with AKP and Sparse-THI over standard CKKS. All methods use the same ring dimension N.

We select parameters targeting 128-bit classical security following the FHE security guidelines [BCC+24]. The main secret key follows the uniform ternary distribution, and the error distribution has standard deviation $\sigma = 3.2$. We use sparse-secret encapsulation [BTH22] during modulus raising, with an ephemeral secret key of Hamming weight 32. For functional bootstrapping, we choose the input scaling factor Δ approximately equal to q_0, as specified in Table 1. Table 1 summarizes the parameter choices.

Table 1: The CKKS scheme parameter sets used for the experiments. Three levels are available after bootstrapping.

Experiment	Base	StC	EvalMod / EvalExp	EvalLUT	CtS	P	$\log_2(QP)$
Boot	$60 + 3 \cdot 40$	$\ell_{\mathrm{StC}} \cdot 40$	$8 \cdot 60$	–	$\ell_{\mathrm{CtS}} \cdot 56$	$3 \cdot 60$	$840 + \ell_{\mathrm{CtS}} \cdot 56$
FBS $(3+4)$	$4 \cdot 47$	$3 \cdot 47$	$53 + 8 \cdot 50$	$\ell_{\mathrm{LUT}} \cdot 47$	$4 \cdot 56$	$3 \cdot 60$	$1045 + \ell_{\mathrm{LUT}} \cdot 47$

We evaluate the linear transformations using the collapsed matrix decomposition and BSGS method of Section 3.

5.2 Bootstrapping over the Conjugate-Invariant Ring

We compare our native bootstrapping method with bootstrapping through standard CKKS using the same N real values sampled independently and uniformly from $[-1, 1]$. For the standard CKKS approach, the total latency includes the conversions between the conjugate-invariant and standard CKKS rings. We measure the output precision as $\mathrm{Prec} = -\log_2 \|\widehat{\mathbf{m}} - \mathbf{m}\|_\infty$, where $\mathbf{m}$ and $\widehat{\mathbf{m}}$ denote the input and bootstrapped slot vectors, respectively.

We consider different total numbers of levels $L = \ell_{\mathrm{StC}} + \ell_{\mathrm{CtS}}$, where ℓ_{StC} and ℓ_{CtS} denote the numbers of factors in StC and CtS, respectively. For each $L \in \{4, 5, 6, 7\}$, we evaluate different allocations of levels between StC and CtS and select the one with the lowest bootstrapping latency. Each factor is evaluated using the diagonal method with BSGS. Table 2 compares the latency and precision of different bootstrapping methods for total linear transformation depths $L = 4, 5, 6, 7$.

Table 2: Bootstrapping latency and precision for different total numbers of levels L. We compare with Lattigo [Tun26], Bae et al. [BCKK25], and Kim et al. [KLN+26].

L	Method	#CI	StC	CtS	EvalMod	Total	Prec.
4	Lattigo	1	5.68 s	32.00 s	3.93 s	42.37 s	20.39 bits
	Lattigo	2	5.71 s	31.87 s	7.80 s	46.32 s	20.30 bits
	Bae et al.	1	56.96 s	261.71 s	1.82 s	321.36 s	16.01 bits
	Kim et al.	1	227.33 s	10.87 s	1.85 s	240.85 s	20.68 bits
	Ours	1	3.59 s	19.20 s	1.91 s	**24.99 s**	20.60 bits
5	Lattigo	1	5.86 s	14.80 s	3.91 s	25.38 s	20.52 bits
	Lattigo	2	5.86 s	14.83 s	7.78 s	29.45 s	20.31 bits
	Bae et al.	1	67.23 s	11.57 s	2.02 s	81.75 s	20.43 bits
	Kim et al.	1	5.61 s	11.51 s	1.94 s	19.89 s	20.70 bits
	Ours	1	3.71 s	10.96 s	1.92 s	**16.91 s**	20.42 bits
6	Lattigo	1	2.63 s	14.98 s	3.92 s	22.37 s	20.46 bits
	Lattigo	2	2.60 s	14.85 s	7.80 s	26.32 s	20.19 bits
	Bae et al.	1	1.79 s	11.49 s	1.92 s	16.14 s	20.47 bits
	Kim et al.	1	5.56 s	7.35 s	1.93 s	15.72 s	20.71 bits
	Ours	1	3.72 s	6.71 s	1.94 s	**12.74 s**	20.80 bits
7	Lattigo	1	2.85 s	14.81 s	3.90 s	22.44 s	20.36 bits
	Lattigo	2	2.86 s	14.75 s	7.78 s	26.49 s	20.13 bits
	Bae et al.	1	1.83 s	8.71 s	2.19 s	13.80 s	20.50 bits
	Kim et al.	1	3.07 s	7.96 s	2.20 s	14.17 s	20.52 bits
	Ours	1	2.03 s	7.34 s	2.33 s	**12.09 s**	20.66 bits

Table 2 compares the latency and precision of different bootstrapping methods for total linear transformation depths $L = 4, 5, 6, 7$. For a single CI ciphertext, our method

consistently outperforms all three baselines. Compared with Lattigo, Bae et al., and Kim et al., our method achieves speedups of 1.50×–1.86×, 1.14×–12.86×, and 1.17×–9.64×, respectively, across the four depths. The gains are particularly pronounced at $L = 4$ and $L = 5$, where the conversion-based methods of Bae et al. and Kim et al. incur substantial overhead. At $L = 6$ and $L = 7$, our method remains the fastest among the single-ciphertext implementations, with total latencies of 12.74 s and 12.09 s, respectively. When Lattigo processes two CI ciphertexts together, its amortized latency is lower than ours at $L = 4$ and $L = 5$, whereas our method achieves lower amortized latency at $L = 6$ and $L = 7$, reducing the per-ciphertext latency from 13.16 s to 12.74 s and from 13.25 s to 12.09 s, respectively.

We also compare the combined latency of StC and CtS in Figure 3 to evaluate the efficiency of the linear transformations.

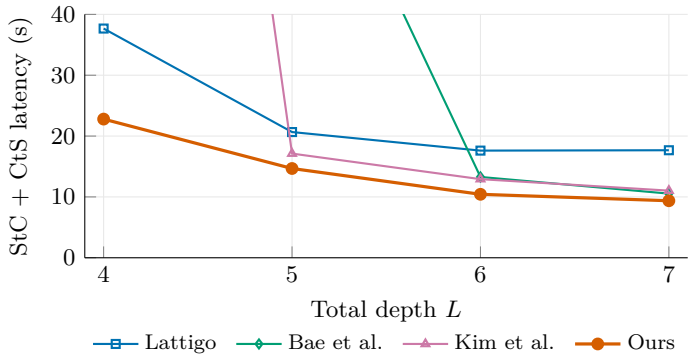


Figure 3: Combined StC and CtS latency for different total linear transformation depths. Values exceeding 40 s are clipped.

5.3 Functional Bootstrapping over the Conjugate-Invariant Ring

We evaluate our two real-valued functional bootstrapping methods over CI-CKKS: CI-AKP, which uses the AKP interpolant with radial correction, and CI-Hermite, which uses our single-cosine Hermite interpolation. We also evaluate CI-Sparse-THI, which implements Sparse-THI over CI-CKKS using simulated complex arithmetic. We compare these three implementations with AKP [AKP25] and Sparse-THI [AKPZ26] over standard CKKS. We consider the identity lookup function $f(m) = m$ over $\mathbb{Z}_{16}$ and the AES S-box over $\mathbb{Z}_{256}$. We use the $3 + 4$ factor configuration for the linear transformations.

We measure the output precision as $\mathrm{Prec}_{\mathrm{LUT}} = -\log_2 \|\widehat{\mathbf{y}} - \mathbf{y}\|_\infty$, where $\mathbf{y} = (f(m_i)/p)_i$ is the exact lookup output and $\widehat{\mathbf{y}}$ is the decoded result. Table 3 reports the nonlinear evaluation time, total latency, and output precision. The nonlinear evaluation includes the phase approximation and Hermite interpolation, while the total latency includes the complete functional bootstrapping procedure.

Table 3 compares the latency, precision, and noise capacity of the functional bootstrapping methods for orders $n = 1, 2, 3$. For $p = 16$, the three CI-CKKS methods achieve comparable noise capacities at each order. Among them, CI-Hermite achieves the lowest latency by evaluating a single real polynomial, whereas CI-AKP requires separate cosine and sine evaluations with radial correction, and CI-Sparse-THI requires additional real ciphertext multiplications to emulate complex arithmetic. Compared with standard CKKS, CI-Hermite achieves amortized speedups of 1.56–1.67× over AKP, while CI-Sparse-THI achieves 1.36–1.43× over Sparse-THI, benefiting from the doubled number of real slots at the same ring dimension.

Table 3: Functional bootstrapping performance for $N = 2^{16}$. For AKP and Sparse-THI, values separated by / correspond to CKKS and CI-CKKS, respectively. The CI-CKKS variant of AKP includes correction.

p	Method	n	Slots	ℓ_{LUT}	EvalLUT (s)	Time (s)	Amort. (ms)	Prec. (bits)	$\log_2 C$
16	AKP	1	$2^{15}/2^{16}$	5/6	9.00/14.12	22.02/31.55	0.672/0.481	21.01/23.63	5.14/6.46
		2	$2^{15}/2^{16}$	5/7	9.05/16.09	21.73/34.37	0.663/0.524	21.62/25.92	8.83/11.71
		3	$2^{15}/2^{16}$	6/7	10.19/16.28	23.51/34.73	0.717/0.530	21.31/25.69	10.28/13.57
	Sparse-THI	1	$2^{15}/2^{16}$	5	9.03/14.27	22.13/31.01	0.675/0.473	28.44/25.52	8.35/6.70
		2	$2^{15}/2^{16}$	6	9.98/15.93	23.30/33.35	0.711/0.509	28.13/25.73	12.68/11.08
		3	$2^{15}/2^{16}$	7	11.36/19.37	25.64/37.81	0.782/0.577	27.81/25.33	14.37/12.58
	CI-Hermite	1	2^{16}	5	9.69	26.70	0.407	25.80	6.26
		2	2^{16}	6	10.53	27.82	0.424	25.77	10.06
		3	2^{16}	6	10.85	28.14	0.429	25.66	12.40
256	AKP	1	$2^{15}/2^{16}$	9/10	16.82/29.08	33.58/52.17	1.025/0.796	16.55/16.91	-1.37/-1.19
		2	$2^{15}/2^{16}$	9/11	18.12/32.41	34.69/56.90	1.059/0.868	17.11/20.05	1.51/3.47
		3	$2^{15}/2^{16}$	10/11	21.78/34.28	39.65/58.73	1.210/0.896	16.61/20.05	2.60/5.18
	Sparse-THI	1	$2^{15}/2^{16}$	9	16.35/31.45	33.13/53.09	1.011/0.810	27.58/23.94	3.58/1.76
		2	$2^{15}/2^{16}$	10	18.97/38.98	36.67/62.70	1.119/0.957	27.24/24.54	7.63/5.83
		3	$2^{15}/2^{16}$	11	21.69/45.55	40.31/69.55	1.230/1.061	26.86/23.11	8.15/6.74
	CI-Hermite	1	2^{16}	9	20.12	41.75	0.637	12.98	-4.19
		2	2^{16}	10	22.98	46.82	0.714	17.90	-0.36
		3	2^{16}	10	24.19	47.26	0.721	19.55	3.79

For $p = 256$, CI-Sparse-THI achieves the highest precision and noise capacity among the CI-CKKS methods. At order 3, it achieves 3.56 bits higher precision and 2.95 bits higher noise capacity than CI-Hermite. The Chebyshev nodes of CI-Hermite become increasingly close near the endpoints as p grows, making the interpolation more sensitive to cosine approximation errors. However, CI-Hermite achieves the lowest latency by evaluating only a single real polynomial.

The correction significantly improves the noise capacity of CI-AKP. As the interpolation order increases from 1 to 3, the noise capacity increases from 6.46 to 13.57 bits for $p = 16$ and from -1.19 to 5.18 bits for $p = 256$. In particular, the negative noise capacity at $p = 256$ and order 1 indicates that the evaluation does not achieve net noise reduction, whereas higher-order correction restores effective noise cleaning. At $p = 256$ and order 3, CI-AKP achieves higher noise capacity than CI-Hermite but remains below CI-Sparse-THI. These improvements require additional multiplicative levels and evaluation time.

6 Conclusion

This work designs a sparse real FFT factorization for CoeffToSlot/SlotToCoeff in CI-CKKS, thereby solving the open problem of designing a native bootstrapping for conjugate-invariant CKKS. Comparable to the batched approach in Lattigo, it has comparable throughput at roughly halved latency and is therefore more flexible. Compared to the methods by Bae et al. and Kim et al., our method avoids the additional layer consumption required by masking, thereby showing better efficiency when the number of layers spent on StC/CtS is fixed. For single-ciphertext bootstrapping, it also achieves a 1.14–12.86× speedup across the evaluated baseline methods and total linear transformation depths.

Another contribution of this work is the exploration of realizing functional bootstrapping in CI-CKKS. Existing complex-valued interpolation methods can be implemented over CI-CKKS by simulating complex arithmetic with pairs of real ciphertexts. We develop two real-valued methods: CI-AKP with radial correction and CI-Hermite based on single-cosine

Hermite interpolation. CI-AKP and CI-Hermite achieve 1.22–1.40× and 1.48–1.68× the throughput of AKP over standard CKKS, respectively. The CI-Sparse-THI implementation achieves 1.16–1.43× the throughput of Sparse-THI over standard CKKS.

References

[AAB+23] Ehud Aharoni, Allon Adir, Moran Baruch, Nir Drucker, Gilad Ezov, Ariel Farkash, Lev Greenberg, Ramy Masalha, Guy Moshkowich, Dov Murik, Hayim Shaul, and Omri Soceanu. HeLayers: A tile tensors framework for large neural networks on encrypted data. *PoPETs*, 2023(1):325–342, January 2023. doi:10.56553/popets-2023-0020.

[AKP25] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using CKKS. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 304–337. Springer, Cham, August 2025. doi:10.1007/978-3-032-01881-6_10.

[AKPZ26] Andreea Alexandru, Andrey Kim, Yuriy Polyakov, and Hongren Zheng. Sparse hermite interpolation method for discrete-CKKS functional bootstrapping. In *Advances in Cryptology – ASIACRYPT 2026*. Springer, 2026. To appear. URL: https://eprint.iacr.org/2026/1026.

[BCC+22] Youngjin Bae, Jung Hee Cheon, Wonhee Cho, Jaehyung Kim, and Taekyung Kim. META-BTS: Bootstrapping precision beyond the limit. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 223–234. ACM Press, November 2022. doi:10.1145/3548606.3560696.

[BCC+24] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, Xianhui Lu, Carsten Maple, Alberto Pedrouzo-Ulloa, Rachel Player, Yuriy Polyakov, Luis Antonio Ruiz Lopez, Yongsoo Song, and Donggeon Yhee. Security guidelines for implementing homomorphic encryption. *CiC*, 1(4):26, 2024. doi:10.62056/anxra69p1.

[BCCW19] Fabian Boemer, Anamaria Costache, Rosario Cammarota, and Casimir Wierzynski. nGraph-HE2: A high-throughput framework for neural network inference on encrypted data. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 45–56. ACM, 2019. doi:10.1145/3338469.3358944.

[BCH+24] Youngjin Bae, Jung Hee Cheon, Guillaume Hanrot, Jai Hyun Park, and Damien Stehlé. Plaintext-ciphertext matrix multiplication and FHE bootstrapping: Fast and fused. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part III*, volume 14922 of *LNCS*, pages 387–421. Springer, Cham, August 2024. doi:10.1007/978-3-031-68382-4_12.

[BCKK25] Youngjin Bae, Jung Hee Cheon, Minsik Kang, and Taeseong Kim. High-throughput AES transciphering using CKKS: Less than 1ms. In *Proceedings of the 13th Workshop on Encrypted Computing & Applied Homomorphic Computing*, pages 1–12. ACM, 2025. doi:10.1145/3733811.3767314.

[BCKS24] Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. Bootstrapping bits with CKKS. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 94–123. Springer, Cham, May 2024. doi:10.1007/978-3-031-58723-8_4.

[BFS+26] Song Bian, Yunhao Fu, Ruiyu Shen, Haowen Pan, Anyu Wang, and Zhenyu Guan. High-precision functional bootstrapping for CKKS from fourier extension. In Joan Daemen and Emmanuel Thomé, editors, *EUROCRYPT 2026, Part IV*, volume 16545 of *LNCS*, pages 274–303. Springer, Cham, May 2026. doi:10.1007/978-3-032-25327-9_10.

[BKSS24] Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping small integers with CKKS. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part I*, volume 15484 of *LNCS*, pages 330–360. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0875-1_11.

[BMTH21] Jean-Philippe Bossuat, Christian Mouchet, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 587–617. Springer, Cham, October 2021. doi:10.1007/978-3-030-77870-5_21.

[BTH22] Jean-Philippe Bossuat, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022*, volume 13269 of *LNCS*, pages 521–541. Springer, Cham, June 2022. doi:10.1007/978-3-031-09234-3_26.

[CCS19] Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part II*, volume 11477 of *LNCS*, pages 34–54. Springer, Cham, May 2019. doi:10.1007/978-3-030-17656-3_2.

[CHH18] Jung Hee Cheon, Kyoohyung Han, and Minki Hhan. Faster homomorphic discrete Fourier transforms and improved FHE bootstrapping. Cryptology ePrint Archive, Report 2018/1073, 2018. URL: https://eprint.iacr.org/2018/1073.

[CHK+18] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 360–384. Springer, Cham, April / May 2018. doi:10.1007/978-3-319-78381-9_14.

[CHKS25] Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. SHIP: A shallow and highly parallelizable CKKS bootstrapping algorithm. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part III*, volume 15603 of *LNCS*, pages 398–428. Springer, Cham, May 2025. doi:10.1007/978-3-031-91131-6_14.

[CK25] Jean-Sébastien Coron and Robin Köstler. Low-latency bootstrapping for CKKS using roots of unity. Cryptology ePrint Archive, Report 2025/651, 2025. URL: https://eprint.iacr.org/2025/651.

[CKKL24] Heewon Chung, Hyojun Kim, Young-Sik Kim, and Yongwoo Lee. Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption. Cryptology ePrint Archive, Report 2024/274, 2024. URL: https://eprint.iacr.org/2024/274.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017. doi:10.1007/978-3-319-70694-8_15.

[CKSS25] Hyeongmin Choe, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Leveraging discrete CKKS to bootstrap in high precision. In Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier, editors, *ACM CCS 2025*, pages 1083–1097. ACM Press, October 2025. doi:10.1145/3719027.3765080.

[CS25] Jean-Sébastien Coron and Tim Seuré. PaCo: Bootstrapping for CKKS via partial CoeffToSlot. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part VII*, volume 16251 of *LNCS*, pages 36–67. Springer, Singapore, December 2025. doi:10.1007/978-981-95-5122-4_2.

[CWCW25] Ray-I Chang, Chia-Hui Wang, Yen-Ting Chang, and Lien-Chen Wei. Approximate homomorphic encryption for MLaaS by CKKS with operation-error-bound. *Computers, Materials & Continua*, 85(1):503–518, 2025. doi:10.32604/cmc.2025.068516.

[DAP+26] Jules Dumezy, Andreea Alexandru, Yuriy Polyakov, Pierre-Emmanuel Clet, Olive Chakraborty, and Aymen Boudguiga. Evaluating larger lookup tables using CKKS. *IACR TCHES*, 2026(1):559–591, 2026. doi:10.46586/tches.v2026.i1.559-591.

[DMPS24] Nir Drucker, Guy Moshkowich, Tomer Pelleg, and Hayim Shaul. BLEACH: Cleaning errors in discrete computations over CKKS. *Journal of Cryptology*, 37(1):3, January 2024. doi:10.1007/s00145-023-09483-1.

[EGR25] Austin Ebel, Karthik Garimella, and Brandon Reagen. Orion: A fully homomorphic encryption framework for deep learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 734–749. ACM, 2025. doi:10.1145/3676641.3716008.

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178. ACM Press, May / June 2009. doi:10.1145/1536414.1536440.

[HS18] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in HElib. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 93–120. Springer, Cham, August 2018. doi:10.1007/978-3-319-96884-1_4.

[JM22] Charanjit S. Jutla and Nathan Manohar. Sine series approximation of the mod function for bootstrapping of approximate HE. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 491–520. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-06944-4_17.

[KLN+26] Taeseong Kim, Jonghoo Lee, Taeyeong Noh, Jung Hee Cheon, and Guillaume Hanrot. Latency-aware, high-throughput homomorphic AES evaluation with CKKS. Cryptology ePrint Archive, Paper 2026/1209, 2026. URL: https://eprint.iacr.org/2026/1209.

[KN25] Jaehyung Kim and Taeyeong Noh. Modular reduction in CKKS. *CiC*, 2(2):17, 2025. doi:10.62056/aevur-iuc.

[KPK+22] Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. EvalRound algorithm in CKKS bootstrapping. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 161–187. Springer, Cham, December 2022. doi:10.1007/978-3-031-22966-4_6.

[KS19] Duhyeong Kim and Yongsoo Song. Approximate homomorphic encryption over the conjugate-invariant ring. In Kwangsu Lee, editor, *ICISC 18*, volume 11396 of *LNCS*, pages 85–102. Springer, Cham, November 2019. doi:10.1007/978-3-030-12146-4_6.

[LLK+22] Yongwoo Lee, Joon-Woo Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and HyungChul Kang. High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 551–580. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-06944-4_19.

[NHY+25] Chao Niu, Zhicong Huang, Zhaomin Yang, Yi Chen, Liang Kong, Cheng Hong, and Tao Wei. XBOOT: Free-XOR gates for CKKS with applications to transciphering. *IACR TCHES*, 2025(4):118–144, 2025. doi:10.46586/tches.v2025.i4.118-144.

[PPP+26] Jaiyoung Park, Sejin Park, Jai Hyun Park, Jung Ho Ahn, Jung Hee Cheon, Guillaume Hanrot, Jung Woo Kim, Minje Park, and Damien Stehlé. Scaling up FHE-based privacy-preserving ML: Higher throughput, longer inputs for LLama-3-8B. arXiv:2601.18511, 2026. URL: https://arxiv.org/abs/2601.18511, arXiv:2601.18511.

[PS73] Michael S. Paterson and Larry J. Stockmeyer. On the number of nonscalar multiplications necessary to evaluate polynomials. *SIAM Journal on Computing*, 2(1):60–66, 1973. URL: https://epubs.siam.org/doi/10.1137/0202007, doi:10.1137/0202007.

[Rah22] Yogachandran Rahulamathavan. Privacy-preserving similarity calculation of speaker features using fully homomorphic encryption. arXiv:2202.07994, 2022. URL: https://arxiv.org/abs/2202.07994, arXiv:2202.07994.

[Tun26] Tune Insight. Lattigo, version 6.2.0. Software source code, 2026. Conjugate-invariant CKKS bootstrapping; accessed September 30, 2026. URL: https://github.com/tuneinsight/lattigo/tree/v6.2.0/circuits/ckks/bootstrapping.